

MLP pour MNIST et moteur de différentiation automatique

Rapport de projet

Master CHPS – UVSQ 2025 - 2026

AUTEURS

Jianye Shi
jianye.shi@ens.uvsq.fr

Hao Qian
hao.qian@ens.uvsq.fr

Xiang Bian
xiang.bian@ens.uvsq.fr

Abdenmour Boulmis
abdenmour.boulmis@ens.uvsq.fr

ENCADREMENT

Aurélien Delval
Enseignant-chercheur

Table des matières

1	Introduction	1
1.1	Contexte du projet et objet d'étude	1
1.2	Base initiale et extensions de cette étape	1
1.3	Périmètre du rapport et organisation	1
2	Optimisation de l'opérateur de multiplication de matrices GEMM	2
2.1	Environnement expérimental	2
2.2	Budget de threads et protocole expérimental	2
2.3	Blocking et packing avec OpenMP	2
2.4	Architecture GotoBLAS	3
2.4.1	Modèle de blocking à trois niveaux	3
2.4.2	Conception du micro-kernel	4
2.5	Réglage mono-thread de GotoBLAS : sélection du micro-kernel AVX2	4
2.5.1	Stratégie de recherche pour N_r et M_r	5
2.5.2	Modèle cache et stratégie de recherche pour k_c	5
2.5.3	Résultats de sélection pour m_c et n_c	5
2.5.4	Workloads de test et protocole de recherche	5
2.5.5	Résultats du réglage	5
2.5.6	Configuration AVX2 finale et aperçu d'AVX-512	6
2.6	Stratégie parallèle multi-threads et réglage des paramètres pour GotoBLAS	7
2.6.1	Réglage des paramètres avec prise en compte des threads	7
2.6.2	Expérience de strong scaling	7
2.7	Principe d'implémentation Unrolled+Prefetch	8
2.8	Expérience complémentaire de montée en charge sur matrices carrées fixes	8
3	Fonction de perte, optimiseurs et boucle d'entraînement	9
4	Implémentation et optimisation de l'opérateur de convolution	11
4.1	Implémentation initiale de la couche de convolution	11
4.2	Parallélisation de la couche de convolution et stratégie d'optimisation	11
4.3	Stratégie de découpage pour la parallélisation	11
4.4	Détails d'implémentation et choix d'ingénierie	12
4.5	Protocole expérimental et configuration de comparaison	12
4.6	Analyse des résultats : gains, limites et déplacement des goulots d'étranglement	13
5	Implémentation de l'architecture CNN	13
5.1	Principe général d'implémentation	13
5.2	Structure de type LeNet	13
5.3	Flux de données avant et propagation des shapes	14
5.4	Couche de pooling et tête de classification entièrement connectée	14
5.5	Abstraction d'interface et organisation du code	15
5.6	Choix de conception et état actuel	15
5.7	Vérification à l'exécution	15
5.8	Extension vers une structure ResNet	16
5.8.1	Structure et stratégie de sous-échantillonnage	16
5.8.2	BasicBlock	16
5.8.3	Résumé	17
6	Recherche automatique d'hyperparamètres avec Optuna	17

7	Portage vers Tiny-ImageNet et pipeline d'entrée	17
7.1	Motivation du portage et caractéristiques des données	18
7.2	Organisation du pipeline d'entrée	18
7.3	État actuel de l'entraînement et principales limites	18
7.4	Périmètre actuel et suite du travail	18
8	Entraînement distribué, synchronisation des gradients et réorganisation du runtime	18
8.1	Parallélisme de données synchrone et agrégation des gradients	18
8.2	Du paramètre par paramètre au gradient bucketing	20
8.3	Chevauchement entre communication et calcul	20
8.4	Compression des gradients : chemin d'implémentation et résultats intermédiaires	20
8.5	Expérience de montée en charge MPI	21
8.6	Résultats de profiling : décomposition des temps de communication	21
8.7	Autograd : du backward implicite vers un moteur d'exécution explicite	21
8.8	Refonte structurelle d'un Engine explicite	22
8.9	Réduction du coût sur le chemin chaud d'Autograd	22
	Références	22
	Glossaire	23
A	Environnement expérimental GEMM, workloads et protocole de recherche	23
A.1	Environnement expérimental	23
A.1.1	Plateforme matérielle	23
A.1.2	Environnement logiciel	23
A.1.3	Protocole de mesure	23
A.2	Workloads utilisés pour le réglage mono-thread	24
A.3	Protocole de recherche en deux tours	24
B	Diagnostic des coûts d'ordonnancement et de synchronisation en configuration 16 threads	24
C	Complément sur le filtrage des paramètres OpenMP blocked / packing	26
D	Dérivation de l'ensemble de candidats AVX2 pour les blocked sizes	27
D.1	Dérivation de m_r et n_r liée à l'implémentation	27
D.2	Modèle de cache et génération des candidats pour k_c	28
D.3	Génération des candidats de base mono-thread pour m_c/n_c	30
E	Synthèse des résultats de réglage AVX2 en mono-thread	33
F	Complément sur la heatmap de réglage thread-aware $kCacheBlockM/N_c$	33
G	Tableau récapitulatif de strong scaling	34

1 Introduction

1.1 Contexte du projet et objet d'étude

La mise en oeuvre d'un système de réseaux de neurones ne se limite pas à l'architecture du modèle. Elle couvre aussi le runtime d'entraînement, les noyaux de calcul numérique et le pipeline d'entrée. Pour une tâche de classification d'images, développer depuis zéro un programme de réseau de neurones entraînable permet d'étudier, sur une base de code unique, les relations entre propagation avant, rétropropagation, mise à jour des paramètres et exécution expérimentale, tout en conservant un environnement maîtrisé pour les mesures de performance et les comparaisons d'implémentation.

Le présent rapport porte sur un système expérimental de réseaux de neurones implémenté en C++. Le système couvre actuellement deux familles de modèles, les perceptrons multicouches (MLP, multi-layer perceptron) et les réseaux convolutifs (CNN, convolutional neural network), ainsi qu'un runtime de différentiation automatique, des backends de multiplication de matrices et de convolution, une interface de chargement des données et un mécanisme de synchronisation pour les expériences d'entraînement distribué. L'analyse porte sur l'organisation de ces composants dans le périmètre actuel, leurs axes d'extension et leur usage expérimental, sans les présenter comme un framework générique de deep learning ni comme une plateforme d'entraînement stabilisée et exhaustive.

1.2 Base initiale et extensions de cette étape

Le travail préparatoire a déjà établi une base d'entraînement de perceptron multicouche sur MNIST [1], incluant la différentiation automatique en mode reverse, un graphe de calcul explicite, un noyau élémentaire de calcul matriciel et une boucle d'entraînement complète. Cet ensemble fournit une base fonctionnelle exécutable et sert de point de départ commun pour comparer plusieurs implémentations de calcul, étendre le chemin convolutif et observer les coûts du runtime.

Sur cette base, cette étape élargit le système, initialement limité à un prototype MLP/MNIST, en un support expérimental capable d'accueillir davantage de variantes techniques. Les extensions principales sont les suivantes : ajout d'un chemin CNN configurable pour les expériences sur modèles convolutifs ; comparaison et optimisation d'implémentations autour de la multiplication de matrices générale (GEMM, general matrix multiplication) et des opérateurs liés à la convolution ; réorganisation du runtime de différentiation automatique afin de distinguer plus clairement les coûts du framework et ceux des opérateurs ; intégration d'un chemin Tiny-ImageNet pour évaluer l'adaptation de l'implémentation actuelle à des entrées image plus lourdes et à une chaîne d'entraînement plus exigeante ; enfin, ajout d'un chemin expérimental de parallélisme de données synchrone fondé sur MPI (message passing interface) afin d'examiner l'organisation et les limites de la synchronisation distribuée des gradients dans l'implémentation actuelle.

1.3 Périmètre du rapport et organisation

La suite du rapport développe ces extensions dans l'ordre suivant. La première partie traite des implémentations GEMM et de leur méthode de comparaison, puis précise la configuration de calcul matriciel réutilisée dans les expériences suivantes. La deuxième présente le périmètre d'implémentation du chemin convolutif et de la structure CNN dans le code actuel. La troisième décrit l'intégration du jeu de données Tiny-ImageNet, le pipeline d'entrée et l'organisation du code de support à l'entraînement. La dernière aborde le chemin de synchronisation distribuée fondé sur MPI ainsi que la réorganisation du runtime de différentiation automatique. Les annexes complètent l'exposé par des expériences de diagnostic et des dérivations de paramètres utilisées

pour appuyer l'analyse et l'interprétation des résultats.

2 Optimisation de l'opérateur de multiplication de matrices GEMM

Dans le système actuel, GEMM (general matrix multiplication) constitue l'opérateur de calcul bas niveau central. Il intervient dans les couches entièrement connectées du MLP et est aussi réutilisé à plusieurs reprises dans le schéma convolutif `im2col + matmul`. L'efficacité de son implémentation influe donc directement sur le temps d'exécution des opérateurs, puis sur le coût global des expériences CNN, Tiny-ImageNet et d'entraînement distribué.

Ce chapitre examine plusieurs chemins GEMM implémentés ou intégrés dans le projet : `blas` (c'est-à-dire l'implémentation OpenBLAS utilisée dans le projet), `omp`, `omp_blocked`, `omp_blocked_packab`, `omp_gotoblas_avx2` et `omp_gotoblas_avx512`. L'objectif n'est pas seulement de comparer ces backends, mais d'adapter et d'optimiser le workload GEMM sur la machine cible à l'aide du parallélisme OpenMP, du blocking cache, du packing, puis d'une organisation hiérarchique de type GotoBLAS associée à des micro-kernels SIMD.

Les questions traitées sont les suivantes : quel gain apporte le parallélisme OpenMP de base, dans quelle mesure le blocking cache et le packing améliorent les accès mémoire, si l'organisation hiérarchique de type GotoBLAS est adaptée au workload du projet, et si les micro-kernels AVX2/AVX-512 restent stables selon la forme des matrices et le nombre de threads. Les résultats servent principalement à fixer la configuration `matmul` réutilisée dans les expériences ultérieures.

2.1 Environnement expérimental

Sauf mention contraire, les expériences de ce chapitre utilisent l'environnement matériel, la configuration logicielle, le protocole de mesure et les réglages d'affinité CPU décrits en annexe A. Les comparaisons GEMM sont limitées par défaut à 1 à 8 threads physiques, avec `OMP_PROC_BIND=true` et `OMP_PLACES=cores`. Seules les expériences de contrôle destinées au diagnostic d'anomalies liées au SMT/hyper-threading utilisent séparément une configuration à 16 threads.

2.2 Budget de threads et protocole expérimental

Avant de comparer les implémentations GEMM, une série d'expériences de contrôle sur le SMT a servi à fixer le budget de threads. La conclusion est qu'une configuration à 16 threads (SMT) dégrade les performances à la fois avec OpenBLAS et avec l'implémentation OpenMP de base. La cause principale est l'amplification des coûts d'ordonnancement et d'attente de synchronisation sur des unités d'exécution physiques partagées par l'hyper-threading (voir annexe B).

En conséquence, toutes les expériences suivantes sont limitées à 1 à 8 threads physiques et imposent `OMP_PROC_BIND=true` ainsi que `OMP_PLACES=cores`, afin d'écarter des comparaisons entre backends les fluctuations d'ordonnancement observées sous SMT. La comparaison de montée en charge avec contraintes d'affinité figure en annexe B.

2.3 Blocking et packing avec OpenMP

La référence `omp` désigne ici l'implémentation GEMM OpenMP de base du projet, sans blocking ni packing. Cette version sert de point de comparaison pour les variantes OpenMP suivantes et pour évaluer l'apport du **blocking** et du **packing** dans l'implémentation actuelle.

Du point de vue de la localité temporelle, le blocking vise à réutiliser autant que possible les sous-blocs de A , B et C déjà présents en cache avant leur éviction. Sans cela, des données

encore peu exploitées peuvent être remplacées par de nouveaux accès.

Le packing remplit une fonction différente du simple blocking. Il ne consiste pas seulement à découper les matrices en sous-blocs, mais à recopier les panneaux ou sous-blocs qui seront relus plusieurs fois dans un tampon contigu, réordonné selon le schéma d'accès du micro-kernel. L'objectif est de concentrer autant que possible le jeu de données utile au bloc courant dans le niveau de cache visé : un petit micro-panel packé peut rester de manière stable en L1/L2 pour les relectures du micro-kernel au niveau registre, tandis qu'un panneau packé plus grand peut être dimensionné pour L2 ou le LLC. En pratique, le packing échange une copie supplémentaire contre des accès séquentiels plus réguliers, moins de strides et une résidence en cache plus contrôlée, ce qui réduit les cache / TLB miss et améliore l'utilisation de la bande passante.

Le gain du blocking et du packing dépend aussi de la charge de travail allouée à chaque thread après découpage selon la dimension M . Lorsque M est faible ou que le nombre de threads est élevé, le nombre de lignes attribuées à chaque thread peut devenir insuffisant pour assurer la réutilisation attendue des tiles. Le programme bascule alors plus souvent dans les traitements de bord, ce qui réduit le bénéfice de réutilisation en cache attendu du blocking, du packing et du parallélisme.

Le réglage détaillé et la procédure de sélection pour `omp_blocked`, `omp_blocked_packb` et `omp_blocked_packab` sont regroupés en annexe C. Sur la plateforme étudiée, les résultats indiquent que la meilleure configuration de `omp_blocked` se rapproche d'un blocking résident en L2, tandis que les variantes avec packing présentent un jeu de données optimal plus proche d'une organisation orientée L1.

À partir de ce constat, la suite se concentre sur un chemin de type GotoBLAS plus structuré afin d'évaluer son adaptation à la machine et au workload considérés.

2.4 Architecture GotoBLAS

Le chemin OpenMP packé décrit plus haut met déjà en évidence l'intérêt des optimisations de localité, mais laisse encore une marge claire d'amélioration. L'implémentation suivante adopte une organisation de type GotoBLAS : blocking à trois niveaux, panneaux packés et micro-kernel SIMD. La recherche de paramètres porte sur $(M_r, N_r, K_c, M_c, N_c)$ et prend pour cible le workload GEMM du projet.

2.4.1 Modèle de blocking à trois niveaux

GotoBLAS décompose $C = A \cdot B$ (de taille $M \times K \times N$) en trois niveaux de boucles imbriquées et un micro-kernel interne (figure 1), chaque niveau correspondant à un étage de cache :

Autrement dit, les trois niveaux externes de blocking et de packing visent à compacter le jeu de données et à le maintenir dans le niveau de cache adapté, tandis que le micro-kernel interne maximise, au niveau des registres, la réutilisation de ces données packées.

1. **Couche externe (dimension N_c)** : parcours de la dimension N avec un pas N_c pour charger dans le LLC (cache L3) les panneaux de B de taille $K \times N_c$;
2. **Couche intermédiaire (dimension K_c)** : parcours de la dimension K avec un pas K_c pour charger en cache L2 les panneaux packés de B de taille $K_c \times N_c$;
3. **Couche interne (dimension M_c)** : parcours de la dimension M avec un pas M_c pour charger en cache L1 les panneaux packés de A de taille $M_c \times K_c$;
4. **Micro-kernel (tile registre $M_r \times N_r$)** : opération la plus interne, avec accumulation d'un bloc de C de taille $M_r \times N_r$ dans les registres SIMD.

Ce modèle hiérarchique suppose que chaque niveau produit des tiles suffisamment réguliers. Lorsque M est faible ou que le nombre de threads est élevé, le nombre de lignes réellement attribuées à chaque thread après découpage selon M peut devenir inférieur à la hauteur du micro-kernel M_r . Les macro-blocs et blocs registres sont alors moins bien remplis, et le programme entre

plus souvent dans les traitements de reste (*remainder loop*), ce qui réduit l'efficacité parallèle et la réutilisation cache attendues du blocking à trois niveaux.

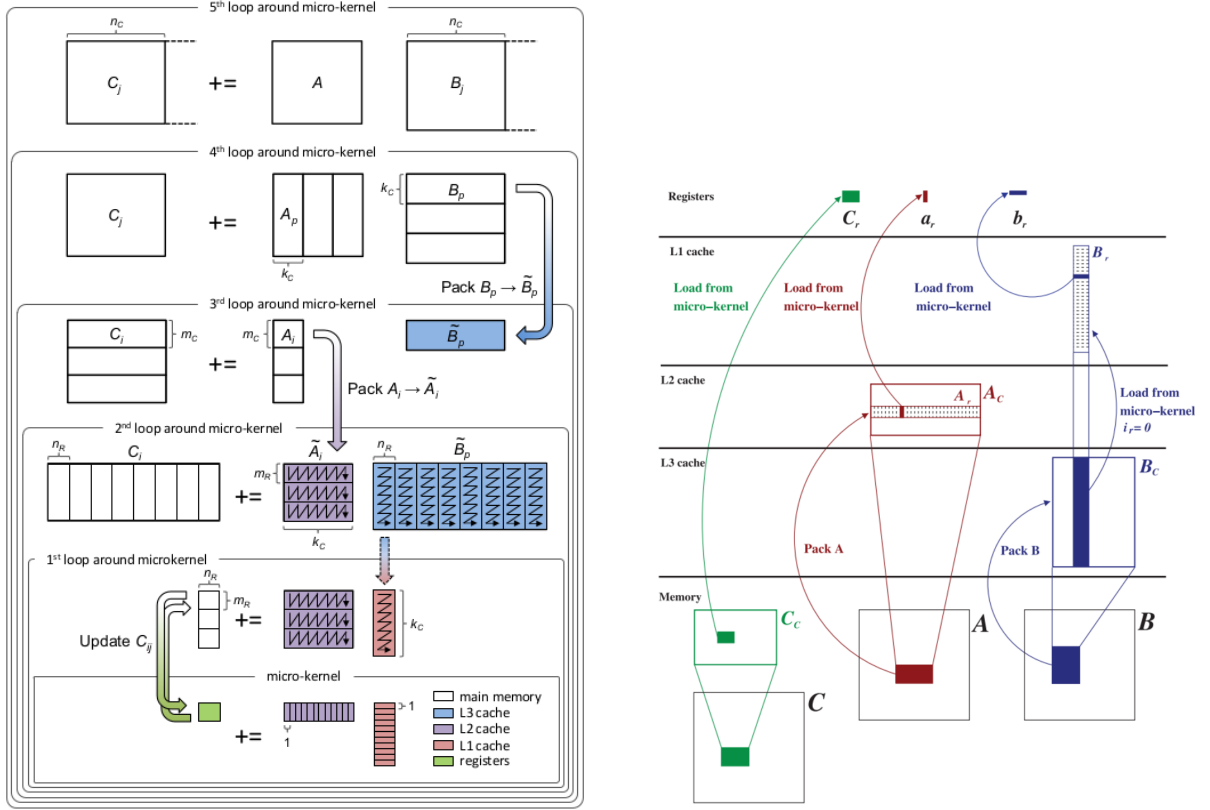


FIGURE 1 – À gauche : schéma de l'organisation en blocs et du micro-kernel de style GotoBLAS/BLIS. À droite : schéma des mouvements de données dans le blocking hiérarchique de GotoBLAS.

Les paramètres de bloc cache sont estimés à partir de la capacité des panneaux : le panneau de A , de taille $M_c \times K_c \times 4$ octets, cible L1 (32 KiB par coeur), et celui de B , de taille $K_c \times N_c \times 4$ octets, cible L2 (1 MiB par coeur). Les valeurs par défaut sont actuellement $K_c = 384$, un nombre de lignes effectif $M_c = 64$ (facteur 8, à partir de $M_r = 8$) et $N_c = 448$. Ces paramètres sont ensuite affinés expérimentalement dans la section 2.5.

2.4.2 Conception du micro-kernel

Le micro-kernel AVX2 en simple précision est organisé par blocs de 8 colonnes selon la dimension N , afin de correspondre à la largeur vectorielle YMM et de limiter les traitements de bord. Dans le chemin `float32`, un registre YMM de 256 bits couvre 8 valeurs `float`. L'implémentation privilégie donc des formes où N_r est multiple de 8.

2.5 Réglage mono-thread de GotoBLAS : sélection du micro-kernel AVX2

La recherche de paramètres s'appuie sur les formes GEMM issues des réseaux de neurones du projet. Comme l'espace combinatoire de $(M_r, N_r, K_c, M_c, N_c)$ est trop large, la sélection est effectuée par étapes : réduction initiale des candidats, puis essais mono-thread sur les formes conservées.

2.5.1 Stratégie de recherche pour N_r et M_r

Pour éviter une exploration exhaustive de (M_r, N_r) et des paramètres de bloc cache, une présélection des formes de micro-kernel est d’abord réalisée à partir de la largeur vectorielle AVX2 en `float32`, des contraintes de masquage de latence et du budget en registres. L’ensemble de candidats retenu est $\{(8, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\}$. La dérivation détaillée est donnée en annexe D.

2.5.2 Modèle cache et stratégie de recherche pour k_c

Pour les six formes AVX2 retenues, une première série de candidats discrets pour K_c est générée pour chaque (M_r, N_r) à partir de la capacité L1 du chemin AVX2 `float32`, des contraintes d’alignement et des contraintes de page footprint / TLB. Le corps du texte ne conserve que les ensembles de candidats transmis au premier tour de sélection mono-thread ; la dérivation complète figure en annexe D.

Forme du noyau	Ensemble candidat pour K_c
8x8	{288, 384}
12x8	{256, 352, 416}
13x8	{224, 320, 384}
4x16	{192, 256, 320}
5x16	{160, 192, 256}
6x16	{128, 160, 224}

TABLE 1 – Première série de candidats K_c pour chaque forme de micro-kernel AVX2

2.5.3 Résultats de sélection pour m_c et n_c

Une fois (M_r, N_r, K_c) fixé, les candidats de base pour M_c et N_c sont eux aussi construits à partir des échelles de cache, puis filtrés par les contraintes de page footprint / TLB. Le premier tour de *coarse screening* couvre toutes les combinaisons (M_r, N_r, K_c) conservées. Le second tour ne retient que cinq couples `KernelShape/Kc` plus compétitifs d’après les résultats mono-thread. Le texte principal donne directement la grille de recherche plus dense utilisée à ce second tour ; le détail de sa construction est reporté en annexe D.

Le nombre total de candidats au second tour est donc de $16 + 12 + 20 + 15 + 12 = 75$, très inférieur à celui d’une recherche non filtrée. Ces combinaisons densifient localement les zones déjà identifiées comme favorables au premier tour afin de stabiliser les conclusions sur les blocked sizes en mono-thread.

2.5.4 Workloads de test et protocole de recherche

Les workloads représentatifs utilisés pour le réglage mono-thread, ainsi que l’organisation détaillée des deux tours de recherche, sont décrits en annexe A. La suite du texte ne conserve que les résultats de sélection et les choix de configuration qui en découlent.

2.5.5 Résultats du réglage

Pour éviter d’insérer dans le corps du rapport l’ensemble des cartes thermiques, un seul workload représentatif de couche entièrement connectée est pris comme exemple : `fc_forward_mainstream_nn / mlp_fc1_b32` ($32 \times 784 \times 128$). La figure 2 présente les GFLOPS absolus obtenus avec différentes formes de micro-kernel AVX2 sur ce workload. Pour chaque *kernel shape*, la figure ne conserve que la meilleure combinaison de paramètres de blocking associée.

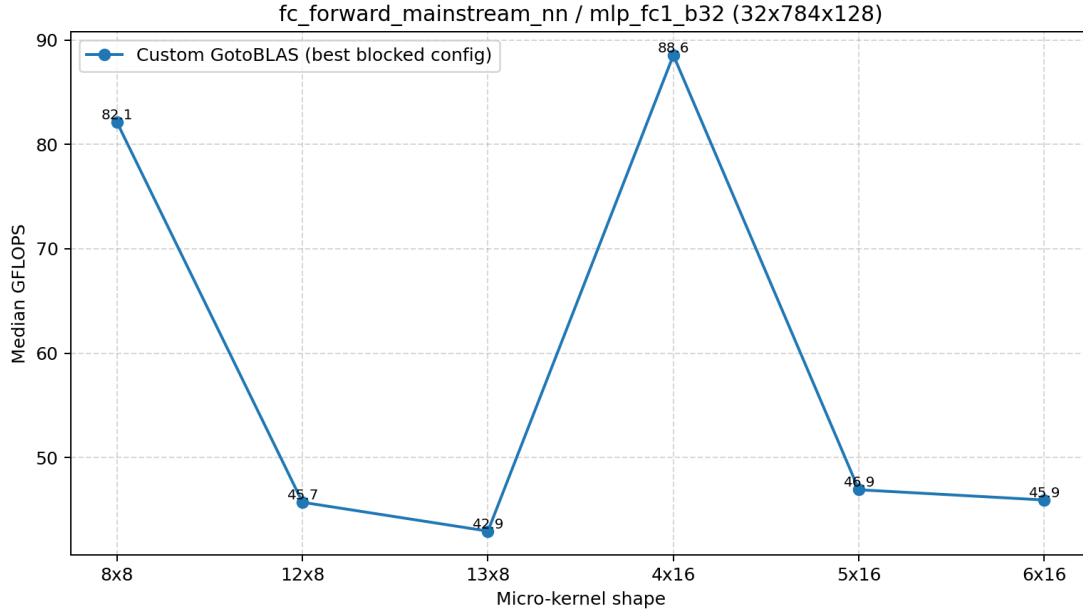


FIGURE 2 – Comparaison mono-thread des GFLOPS pour différentes formes de micro-kernel AVX2 sur le workload de couche entièrement connectée `mlp_fc1_b32`. L'axe horizontal représente la forme du micro-kernel et l'axe vertical la médiane des GFLOPS obtenue avec les meilleurs paramètres de blocking pour cette forme.

La figure 2 montre que, sur ce workload représentatif de couche entièrement connectée, les formes `4x16` et `8x8` dominent nettement. La forme `4x16` atteint environ 88.6 GFLOPS, suivie par `8x8` avec environ 82.1 GFLOPS, tandis que les autres formes restent pour la plupart dans l'intervalle 43 à 47 GFLOPS.

Le récapitulatif des meilleures formes de noyau et des paramètres de bloc associés pour chaque famille de workloads est donné en annexe E.

Les résultats font apparaître deux types d'écart de performance :

Formes courantes ($N \geq 64$) : l'écart avec OpenBLAS se situe entre 10 et 24%, ce qui reflète l'écart entre notre micro-kernel et l'assemblage fortement optimisé d'OpenBLAS, ainsi que le niveau d'adaptation plus fin d'OpenBLAS à certaines formes, par exemple via le padding et les stratégies de prefetch.

Formes à petit N ($N < 32$, comme `fc_head`) : l'écart atteint 66 à 78%. Un N très faible ne permet pas de former un tile registre N_r efficace, ce qui introduit beaucoup de traitements de bord. OpenBLAS dispose de chemins spécialisés pour ces formes dégénérées, alors que l'implémentation actuelle n'est pas encore optimisée pour ce cas.

2.5.6 Configuration AVX2 finale et aperçu d'AVX-512

Au vu des résultats du réglage mono-thread, les expériences multi-threads suivantes utilisent la configuration AVX2 fixe suivante :

$$\text{avx2_8x8}, \quad K_c = 384, \quad M_c = 64, \quad N_c = 448$$

où $M_c = 64$ correspond au nombre effectif de lignes (constante de code `kCacheBlockM=8`, valeur effective = `kCacheBlockM` × $M_r = 8 \times 8 = 64$). Cette configuration provient de la sélection mono-thread effectuée sur l'ensemble de workloads présenté plus haut.

Le micro-kernel AVX-512 est réglé sur le même chemin de données unifié `float32/Scalar=float` que le reste du projet. Avec des registres ZMM de 512 bits, un vecteur peut contenir 16 valeurs `float`. Dans un micro-kernel tel que `avx512_8x32`, où $N_r = 32$, la dimension N est généralement traitée par blocs de deux vecteurs ZMM. `avx512_8x32` ($M_r = 8$, $N_r = 32$) accumule $8 \times 32 = 256$ éléments par FMA, avec un pic théorique en FLOP/cycle deux fois plus élevé que celui du noyau

AVX2 8x8. La configuration finale retenue est : `avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$ (M_c correspond à 64 lignes effectives; $K_c = 160$ est une valeur issue du réglage expérimental, transmise à l'exécution via `MATMUL_KC=160`, contrairement à la constante compilée `kCacheBlockK=384` du chemin AVX2). La dérivation détaillée des ensembles candidats pour le blocked-size AVX2 est donnée en annexe D.

2.6 Stratégie parallèle multi-threads et réglage des paramètres pour GotoBLAS

Le réglage mono-thread détermine la meilleure forme de noyau et les meilleurs paramètres de bloc. En multi-threads, l'implémentation conserve les valeurs M_c , N_c et K_c fournies à l'exécution comme paramètres de blocking cache, puis introduit à l'étage d'ordonnancement une granularité de tâche dépendante du nombre de threads : le code calcule un `task_m` plus fin sur la dimension ic/M afin de générer davantage de tâches sur les petites matrices et de réduire l'inactivité des threads.

2.6.1 Réglage des paramètres avec prise en compte des threads

Dans le code actuel, la prise en compte du nombre de threads se traduit surtout par la granularité d'ordonnancement, et non par une modification des paramètres de bloc cache eux-mêmes. Une fois dans la région parallèle, le programme calcule donc un `task_m` plus fin selon le nombre de threads, tandis que les expériences de *strong scaling* utilisent uniquement la configuration finale à blocked-size fixe, sans détailler ici les étapes intermédiaires du réglage.

Les expériences de *strong scaling* qui suivent utilisent la configuration fixe suivante : pour le chemin AVX2, `avx2_8x8`, $K_c = 384$, $M_c = 64$, $N_c = 448$; pour le chemin AVX-512, `avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$. Ici, $M_c = 64$ représente le nombre effectif de lignes (constante de code `kCacheBlockM=8`, valeur effective = $8 \times M_r = 8 \times 8 = 64$). Une carte thermique représentative de `kCacheBlockM`/ N_c avec prise en compte des threads est fournie en annexe, figure 10. La zone de meilleures performances y est concentrée sur la ligne `kCacheBlockM=8` (soit $M_c = 64$ effectif), tandis que la direction N_c forme un plateau relativement plat.

2.6.2 Expérience de strong scaling

L'expérience de *strong scaling*, à taille de problème fixe et nombre de threads variable ($T \in \{1, 2, 4, 8\}$), compare trois implémentations : GotoBLAS AVX2, GotoBLAS AVX-512 et OpenBLAS. L'ensemble de test couvre 9 workloads, incluant des formes issues de MLP, de convolutions CNN et de matrices carrées.

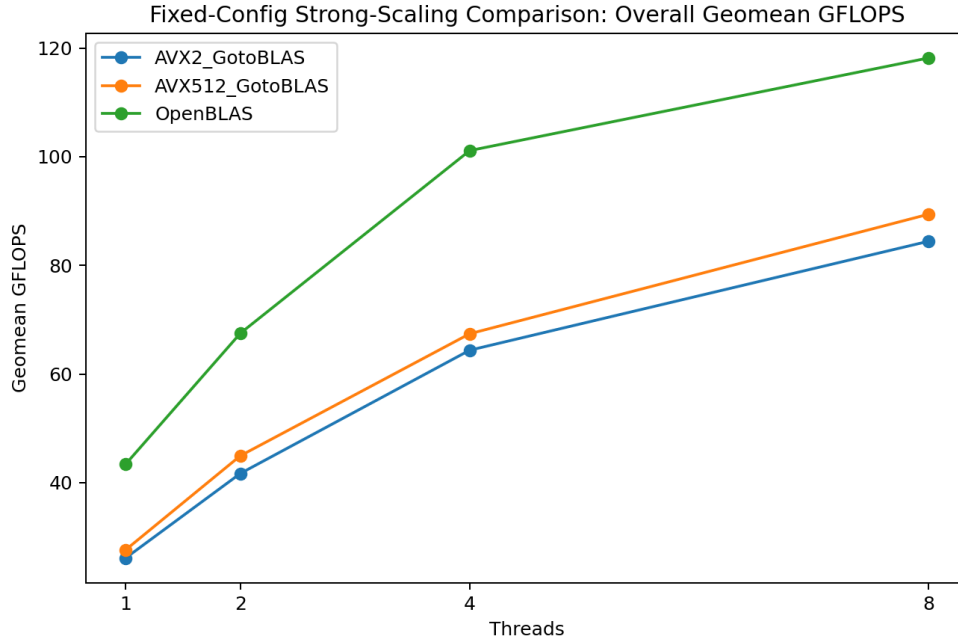


FIGURE 3 – Expérience de strong scaling : moyenne géométrique des GFLOPS pour chaque nombre de threads, calculée sur 9 workloads.

La figure 3 repose sur la moyenne géométrique de 9 workloads à configuration fixe. Elle montre que, sur cette agrégation, OpenBLAS conserve les GFLOPS absolus les plus élevés, tandis que les chemins GotoBLAS AVX2 et AVX512 présentent une montée plus régulière avec le nombre de threads. Selon cette mesure, à 8 threads, les versions GotoBLAS AVX2 et AVX512 atteignent toutes deux un speedup d'environ $3.23\times$, contre environ $2.72\times$ pour OpenBLAS. Les résultats détaillés par workload proviennent de la comparaison *fixed-config strong-scaling*. Sur la majorité des workloads, OpenBLAS atteint des GFLOPS absolus plus élevés ; le chemin GotoBLAS sert surtout à comparer le blocking, le packing et l'organisation OpenMP, tout en restant compétitif sur certaines matrices carrées et sur quelques formes GEMM liées à la convolution. Le tableau récapitulatif du strong scaling est donné en annexe G.

2.7 Principe d'implémentation Unrolled+Prefetch

Après le réglage de base du chemin AVX-512, le code a été étendu par une variante **unrolled+prefetch**. Cette version conserve la structure blocked+packed existante et ajoute seulement du déroulage de boucle (par exemple `GEMM_UNROLL_4`) et du prefetch explicite dans le micro-kernel AVX-512 et dans la phase d'accès aux panneaux packés correspondants. L'objectif est de réduire davantage l'attente mémoire dans la partie la plus interne de l'exécution. Cette variante agit sur le chemin d'accès séquentiel aux panneaux packés A/B , avec le même périmètre statistique que dans les sections précédentes.

2.8 Expérience complémentaire de montée en charge sur matrices carrées fixes

En complément des workloads du projet décrits plus haut, une expérience supplémentaire de montée en charge a été menée sur quatre tailles fixes de matrices carrées : 64×64 , 128×128 , 256×256 et 512×512 . La figure 4 rassemble les résultats de `blas`, `omp`, `omp_blocked`, `omp_blocked_packab`, `omp_gotoblas_avx2` et du chemin AVX-512 unroll+prefetch sur l'intervalle 1–8 threads physiques.

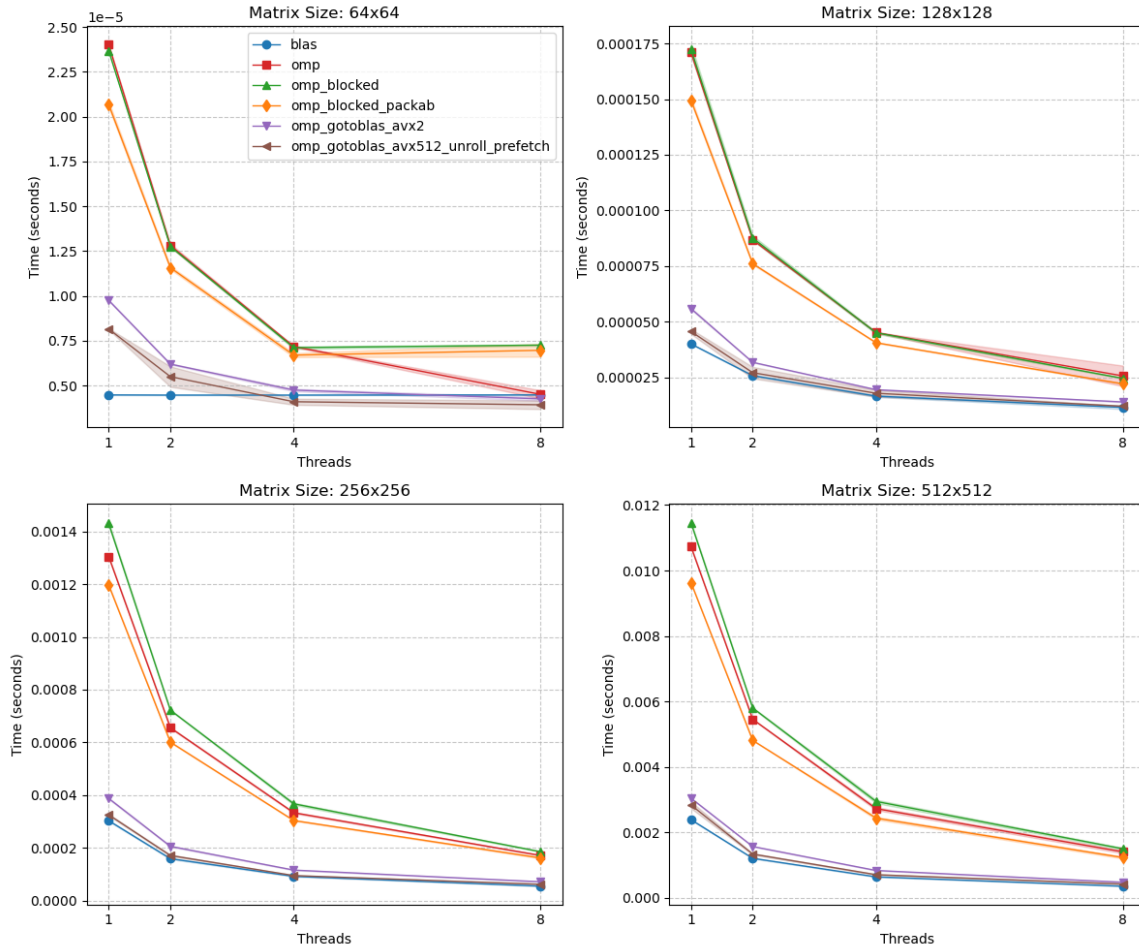


FIGURE 4 – Expérience complémentaire de montée en charge sur matrices carrées fixes avec plusieurs implémentations. Les quatre sous-figures correspondent respectivement à 64×64 , 128×128 , 256×256 et 512×512 ; l'axe vertical représente le temps d'exécution et l'axe horizontal le nombre de threads.

La figure 4 montre que, pour 64×64 , `blas` reste proche d'un comportement quasi mono-thread et varie peu avec le nombre de threads, tandis que les chemins OpenMP n'atteignent généralement ce niveau qu'autour de 4 à 8 threads. Entre 128×128 et 512×512 , les courbes de temps de `omp_gotoblas_avx2` et de `omp_gotoblas_avx512_unroll_prefetch` décroissent plus rapidement; parmi les tailles fixes et l'intervalle 1–8 threads représentés ici, la seconde variante reste en général légèrement meilleure que le chemin AVX2.

3 Fonction de perte, optimiseurs et boucle d'entraînement

Dans le point d'entrée principal de l'entraînement, la fonction de perte est fixée à `CrossEntropyLoss`. Le modèle produit des logits de classe en sortie de la propagation avant; la couche de perte applique en interne un softmax numériquement stable et l'entropie croisée, puis renvoie une perte scalaire pour l'ensemble du batch. Une implémentation de `MSELoss` est également conservée dans le code, mais le chemin de classification utilise uniquement `CrossEntropyLoss`.

Du côté des optimiseurs, l'implémentation actuelle propose trois variantes : `SGD`, `MomentumSGD` et `AdamW`. La figure 5 regroupe les courbes d'entraînement sur 20 epochs pour ces trois optimiseurs appliqués au MLP sur MNIST.

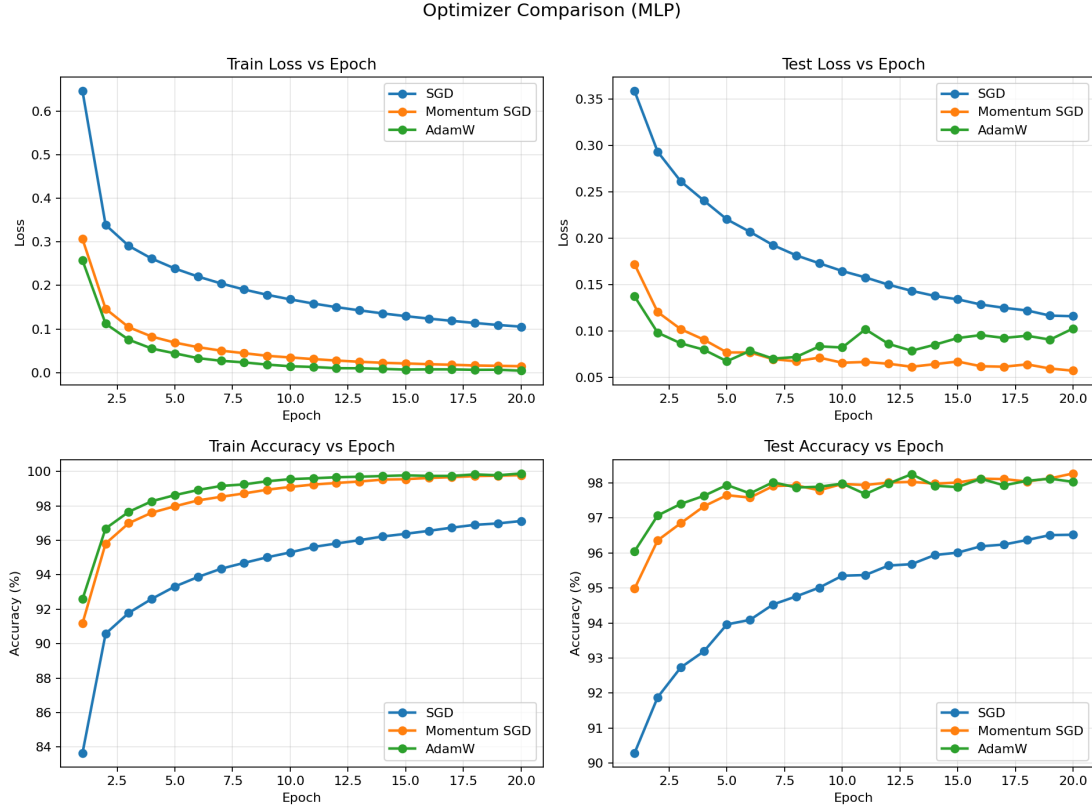


FIGURE 5 – Comparaison des pertes et des précisions d'entraînement/test pour trois optimiseurs sur le MLP MNIST (20 epochs). La précision finale en test est de 96.52% pour SGD, 98.27% pour MomentumSGD et 98.03% pour AdamW.

La figure 5 montre que **SGD** converge plus lentement et reste en retrait en précision finale, tandis que **MomentumSGD** obtient ici la meilleure précision de test ; **AdamW** atteint un niveau proche, avec une convergence plus rapide au début de l'entraînement.

Ces trois variantes correspondent à des mécanismes de mise à jour différents. Pour **SGD**, les paramètres sont mis à jour directement à partir du gradient courant :

$$\theta_{t+1} = \theta_t - \eta g_t.$$

MomentumSGD ajoute un terme de vitesse, qui lisse exponentiellement l'historique des gradients afin de stabiliser la direction de mise à jour :

$$v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1},$$

La correction de Nesterov peut être activée selon la configuration. **AdamW** maintient en plus des estimations du premier et du second moment du gradient et découple le *weight decay* de la mise à jour adaptative, ce qui améliore le comportement de régularisation tout en conservant une convergence stable.

Une fois le graphe de calcul construit, l'optimiseur est sélectionné à partir des paramètres de ligne de commande. À chaque itération d'entraînement, les gradients sont d'abord remis à zéro ; après le backward et, le cas échéant, la synchronisation distribuée, ils sont mis à l'échelle selon la taille du batch global avant la mise à jour des paramètres. **Trainer::runEpoch(...)** porte à lui seul les chemins d'entraînement et d'évaluation : le premier exécute propagation avant, calcul de la perte, rétropropagation et mise à jour des paramètres ; le second conserve uniquement la propagation avant et le calcul des métriques. La fonction de perte, l'optimiseur

et la boucle d'entraînement constituent donc la base commune aux expériences CNN, ResNet et distribuées décrites ensuite.

4 Implémentation et optimisation de l'opérateur de convolution

Cette section introduit l'implémentation de la convolution pour les futurs CNN et se concentre sur l'organisation du chemin principal `im2col`/`GEMM`/`col2im` ainsi que sur ses marges d'optimisation.

4.1 Implémentation initiale de la couche de convolution

`Conv2DLayer` réécrit, dans le backend de référence, la propagation avant d'une convolution en une opération GEMM après `im2col` :

$$\begin{aligned} \text{im2col}(X) &\in \mathbb{R}^{(N \cdot H_{out} \cdot W_{out}) \times (C_{in} \cdot k_H \cdot k_W)}, \\ W &\in \mathbb{R}^{C_{out} \times (C_{in} \cdot k_H \cdot k_W)}, \\ Y &= \text{im2col}(X) \cdot W^T + b. \end{aligned}$$

Ce choix réutilise le chemin `MathOps::matmul`. Dans l'implémentation actuelle, le coût de la convolution se répartit entre `im2col`, GEMM, la réorganisation de la sortie, `dW/dX` et `col2im`. L'analyse de cette section se limite à cette chaîne `im2col`/`GEMM`/`col2im`.

4.2 Parallélisation de la couche de convolution et stratégie d'optimisation

Dans l'implémentation CNN actuelle du projet, la couche de convolution repose sur le chemin `im2col` + GEMM. Cette approche permet de réutiliser un backend de multiplication de matrices déjà optimisé, mais la performance de la couche n'est pas déterminée par GEMM seul. En pratique, le coût total provient principalement de trois composantes : la réorganisation des données par `im2col` à l'étape avant, l'opération de multiplication de matrices elle-même, puis `col2im` et les GEMM liés aux gradients pendant la phase arrière. Ainsi, même avec un GEMM rapide, la couche de convolution peut rester un goulot d'étranglement à cause de la réorganisation des données et des coûts d'ordonnancement des threads.

À partir de ce constat, l'objectif n'est pas d'accélérer un kernel isolé, mais de réorganiser la parallélisation de bout en bout autour du chemin principal de convolution : réduire le surcoût des étapes de réagencement, améliorer la montée en charge multi-threads et limiter les pertes liées au parallélisme sur les petites tâches, sans modifier la sémantique numérique. La comparaison porte donc sur le débit global de la chaîne de convolution, et non sur la vitesse de pointe d'un seul kernel.

4.3 Stratégie de découpage pour la parallélisation

Le premier choix pour paralléliser les passes avant et arrière d'une convolution concerne la dimension de découpage. En théorie, les tâches peuvent être réparties selon la dimension batch, les canaux de sortie, la dimension spatiale de sortie ($H_{out} \times W_{out}$) ou la dimension de réduction. Ces variantes diffèrent nettement du point de vue de l'équilibrage de charge, de la réutilisation cache et du risque d'écritures conflictuelles.

L'implémentation privilégie le principe d'indépendance des éléments de sortie : chaque

thread traite des blocs de sortie disjoints afin d'éviter des verrous ou opérations atomiques supplémentaires. Dans le chemin avant, la phase `im2col` est découpée par échantillons et par blocs de sortie spatiale pour garantir des zones d'écriture disjointes entre threads; la phase GEMM réutilise directement la stratégie parallèle de la multiplication de matrices sous-jacente, sans gestion fine supplémentaire au niveau de la couche de convolution. Dans le chemin arrière, les calculs matricialisés de `dW` et `dX` sont eux aussi organisés autour de blocs de sortie, tandis que `col2im` réduit le risque d'écritures conflictuelles au moyen d'une diffusion par blocs et de protections de bord.

Un découpage plus fin n'apporte pas systématiquement un meilleur résultat. Lorsque la granularité des tâches devient trop faible, les coûts fixes de création de threads, de synchronisation par barrière et de migration d'ordonnancement augmentent fortement; cet effet est particulièrement visible sur des entrées de petite taille comme celles de MNIST. La stratégie de découpage doit donc équilibrer le degré de parallélisme et la densité de calcul par tâche : fournir suffisamment de travail pour exploiter les coeurs, sans fragmenter l'exécution en une multitude de tâches trop courtes.

4.4 Détails d'implémentation et choix d'ingénierie

Sur le plan de l'implémentation, l'optimisation se concentre sur trois points. Le premier consiste à améliorer la continuité des accès mémoire : l'ordre des boucles et les calculs d'indices de `im2col/col2im` sont ajustés pour que les boucles internes parcourent autant que possible des adresses contiguës, ce qui augmente l'utilisation des lignes de cache et réduit le coût des calculs d'adresse. Le deuxième concerne la gestion des tampons temporaires : les buffers intermédiaires sont réutilisés lorsque c'est possible afin de limiter les allocations et désallocations répétées pendant l'entraînement, et donc de réduire les fluctuations de l'allocateur. Le troisième porte sur la stratégie d'ordonnancement des threads : un découpage statique plus stable est privilégié lorsque la charge est régulière, afin de réduire le coût d'ordonnancement du runtime et les migrations de threads.

Ces modifications ont un trait commun : elles réorganisent le chemin d'exécution sans modifier la sémantique mathématique. Les résultats des passes avant et arrière restent donc alignés avec l'implémentation de base, et les optimisations se limitent à l'organisation du calcul, à la disposition des données et à l'ordonnancement parallèle. Cette approche présente deux avantages directs : la vérification de la correction reste simple, car les écarts doivent principalement venir de l'ordre des arrondis en virgule flottante; elle reste également compatible avec le flux actuel d'autodiff et d'optimisation, sans réécriture de la boucle principale d'entraînement.

4.5 Protocole expérimental et configuration de comparaison

Pour évaluer le gain de la parallélisation, les expériences suivent le même protocole reproductible que dans les sections précédentes : environnement matériel et options de compilation fixés, mêmes formes d'entrée et même configuration de batch, puis comparaison entre le chemin convolutif de base et le chemin optimisé. Le nombre de threads est généralement pris dans $\{1, 2, 4, 8\}$ avec une politique d'affinité CPU identique, afin d'éviter que le bruit d'ordonnement ne masque les différences d'implémentation. Chaque série de tests comprend des itérations d'échauffement et des répétitions, la médiane des temps servant de statistique principale, complétée par l'écart-type pour juger de la stabilité des résultats.

Les mesures ne se limitent pas au temps de la seule propagation avant d'une convolution. Elles enregistrent aussi séparément `im2col`, le GEMM convolutif, `col2im` et les étapes liées au backward, afin de distinguer le gain sur l'opérateur lui-même et le gain de bout en bout sur la chaîne complète. Une observation limitée à GEMM tend à surestimer l'effet global de l'optimisation; un découpage des temps met plus clairement en évidence le déplacement des goulots d'étranglement, qu'il provienne du calcul, de la réorganisation des données ou de l'ordonnance-

ment parallèle.

4.6 Analyse des résultats : gains, limites et déplacement des goulots d'étranglement

Les résultats expérimentaux montrent que l'optimisation parallèle de la couche de convolution apporte un gain stable sur les workloads de taille moyenne ou supérieure, et que la durée totale des passes avant et arrière diminue par rapport à la version de base. Cette amélioration ne provient donc pas d'une seule étape, mais d'un progrès coordonné sur l'ensemble de la chaîne `im2col/GEMM/col2im`.

Le domaine de validité est lui aussi clair : lorsque la taille des entrées ou du batch est faible, une partie du gain parallèle est annulée par les coûts fixes d'ordonnancement, et les petits problèmes restent sensibles à un parallélisme trop agressif. L'intérêt de cette optimisation ne se limite donc pas à la réduction du temps de convolution ; elle permet aussi de faire apparaître plus nettement les goulots d'étranglement suivants. Les étapes suivantes portent encore sur le coût de réagencement de la convolution, le chemin backward et l'adéquation entre formes de kernels et workloads.

5 Implémentation de l'architecture CNN

Ce chapitre décrit l'intégration du chemin CNN dans le framework d'entraînement existant. Le code réutilise, via l'interface `NeuralNetwork`, le trainer, les optimiseurs, la différentiation automatique et le chargement des données ; les ajouts se concentrent sur `Conv2D`, `MaxPool2D` et `CNNNetwork`.

5.1 Principe général d'implémentation

L'intégration du CNN conserve le trainer, la fonction de perte, l'optimiseur et le cadre de calcul `Node/Matrix` déjà en place. Une interface unifiée `NeuralNetwork` est ajoutée au niveau de l'abstraction réseau afin que `MLPNetwork` et `CNNNetwork` partagent la même logique d'entraînement. La stratification logicielle reste inchangée : `Matrix` et la multiplication de matrices en bas, `Node/MathOps` au niveau intermédiaire, puis `Conv2DLayer`, `MaxPool2DLayer` et `CNNNetwork` au niveau supérieur.

5.2 Structure de type LeNet

Le chemin CNN actuel utilise une structure de type `LeNet` adaptée à MNIST. Chaque échantillon entre d'abord dans le réseau sous la forme $(N, 784)$, puis est interprété pendant la propagation avant comme une image monochrome de forme $(N, 1, 28, 28)$. Le corps du réseau comprend deux couches de convolution, deux couches de pooling et trois couches entièrement connectées ; les transformations de forme sont résumées dans la figure 6.

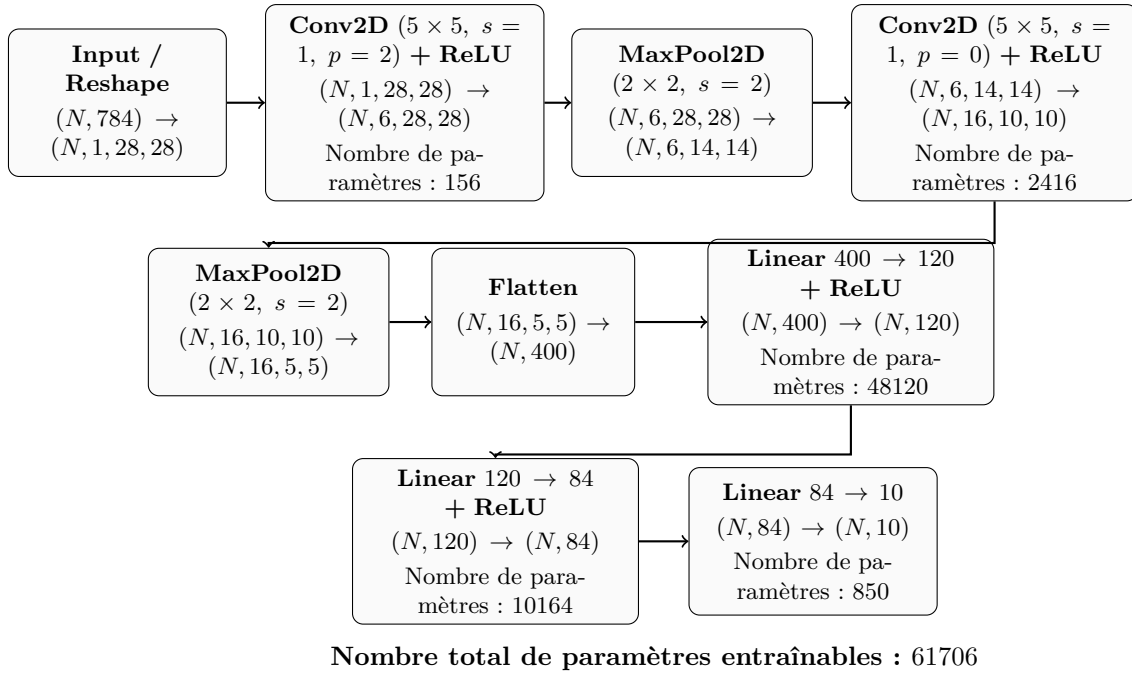


FIGURE 6 – Structure du chemin CNN actuel et évolution des shapes à chaque étape.

L'ensemble du réseau compte environ 61706 paramètres entraînables. Cette échelle correspond au chemin CNN actuel pour MNIST et couvre quatre types de calcul : convolution, pooling, mise à plat des caractéristiques et tête de classification.

5.3 Flux de données avant et propagation des shapes

Le chemin avant de ce réseau suit la figure 6 : l'entrée est d'abord interprétée de $(N, 784)$ vers $(N, 1, 28, 28)$, puis traverse successivement deux convolutions, deux opérations de pooling, une mise à plat des caractéristiques et une tête de classification à trois couches entièrement connectées.

Le choix d'implémentation consiste à conserver **Matrix** comme stockage bas niveau unique, sans introduire de **Tensor4D** distinct. Le projet utilise uniformément des **float** en simple précision ; un tenseur logique à quatre dimensions (N, C, H, W) est aplati en mémoire dans une matrice bidimensionnelle **Matrix**($N, C \times H \times W$), puis réinterprété au moyen des paramètres de forme. Pour un élément (n, c, h, w) , l'indice linéaire s'écrit

$$\text{idx} = c \times H \times W + h \times W + w.$$

Ce choix évite de réécrire un nouveau système de stockage tensoriel et de nouveaux opérateurs bas niveau, tout en permettant à la couche de convolution de réutiliser directement les interfaces matricielles existantes. En contrepartie, chaque couche doit transmettre explicitement ses informations de forme et manipuler avec précaution les opérations de reshape et d'indexation.

5.4 Couche de pooling et tête de classification entièrement connectée

MaxPool2DLayer assure le sous-échantillonnage spatial. L'implémentation actuelle utilise un max pooling ; pendant la passe avant, la position du maximum est enregistrée dans chaque fenêtre, puis le gradient est redistribué à la position correspondante lors de la passe arrière. La couche de pooling ne contient aucun paramètre entraînable.

Après deux convolutions et deux opérations de pooling, la taille de la carte de caractéris-

tiques passe de $(N, 1, 28, 28)$ à $(N, 16, 5, 5)$. Le réseau effectue ensuite directement la mise à plat dans `forward()` pour obtenir $(N, 400)$, puis transmet ce résultat à la tête de classification à trois couches entièrement connectées. Cette mise à plat est traitée en interne comme une opération de reshape.

5.5 Abstraction d’interface et organisation du code

Pour que le `trainer` puisse prendre en charge à la fois le MLP, le CNN de type LeNet et les futures variantes ResNet, l’implémentation actuelle n’introduit pas de boucle d’entraînement distincte pour chaque famille de modèles. Elle ajoute au contraire une abstraction unifiée `NeuralNetwork` au-dessus de la couche réseau. Cette interface ne conserve que les deux capacités réellement nécessaires à la boucle principale : exécuter la propagation avant à partir des entrées et renvoyer l’ensemble des paramètres entraînaibles. Le `trainer`, la fonction de perte, l’optimiseur et la logique de remise à zéro des gradients et de mise à jour des paramètres restent ainsi inchangés ; les différences entre modèles sont cantonnées à leur structure interne.

Dans cette organisation, les couches de convolution, de pooling et les couches entièrement connectées sont considérées comme des composants internes du réseau, et non comme des branches particulières du framework d’entraînement. Que le modèle sous-jacent soit `MLPNetwork`, `CNNNetwork` ou `ResNetNetwork`, le `trainer` manipule toujours la même interface : les données d’entrée passent par `forward()`, puis l’optimiseur reçoit la liste des paramètres à mettre à jour après la rétropropagation. Cette conception découple autant que possible les variations d’architecture de modèle des variations du flux d’entraînement, et permet d’intégrer le réseau convolutif sur la base du pipeline MLP existant sans réécrire tout le cadre d’exécution.

Du point de vue de l’organisation du code, les ajouts suivent principalement la hiérarchie « implémentation des couches $\rightarrow$ composition du réseau $\rightarrow$ point d’entrée d’entraînement » : les couches de convolution et de pooling portent la sémantique locale des opérateurs, `CNNNetwork` et `ResNetNetwork` assemblent les structures concrètes, puis le programme principal choisit, via les arguments de ligne de commande, quel réseau instancier. Les fonctions de perte, optimiseurs, noeuds d’autodiff et backends de calcul matriciel existants restent donc presque inchangés. Les optimisations de convolution, le profiling d’entraînement et le chemin de synchronisation distribuée décrits ensuite s’appuient tous sur ce cadre d’entraînement unifié.

5.6 Choix de conception et état actuel

Le chemin CNN principal repose encore sur une structure de type LeNet et sur un schéma de convolution `im2col + matmul`, avec réutilisation du stockage unifié `Matrix` et du framework d’entraînement existant. L’extension ResNet est déjà intégrée, mais n’a pas encore fait l’objet d’un réglage systématique des hyperparamètres ni de campagnes de benchmark ; des composants plus complets, comme BatchNorm, restent également à développer.

5.7 Vérification à l’exécution

Modèle	Nombre d’epochs	Précision en test
MLP	1	80.10%
MLP	3	87.12%
MLP	5	89.00%
CNN	1	88.82%

TABLE 2 – Comparaison de la précision en test du CNN et du MLP sur MNIST (batch=256, seed=42)

Dans cette comparaison grossière avec ‘batch=256’ et ‘seed=42’, le CNN atteint 88.82% après 1 epoch, soit un niveau proche des 89.00% obtenus par le MLP après 5 epochs. Ce résultat

sert principalement à montrer que le chemin principal du CNN s'exécute désormais de manière stable et atteint, dans cette configuration, un niveau de précision sur MNIST proche de celui du MLP.

5.8 Extension vers une structure ResNet

Après le CNN de type LeNet, le projet a également intégré `ResNetNetwork` afin de préparer des expériences sur des réseaux convolutifs plus profonds. Le principe central de ResNet est la connexion résiduelle : chaque bloc n'apprend plus un mapping complet $H(x)$, mais un résidu $F(x)$ relatif à l'entrée, avec une sortie écrite

$$y = F(x) + x.$$

Cette structure atténue les problèmes de dégradation du gradient dans l'entraînement des réseaux profonds et conserve une bonne aptitude à l'optimisation lorsque la profondeur augmente.

L'extension ResNet suit toujours l'interface unifiée `NeuralNetwork`. Elle se branche donc directement sur le même `trainer`, la même fonction de perte, le même optimiseur et le même système de différentiation automatique, comme modèle optionnel dans le flux d'entraînement existant. La boucle d'entraînement ne nécessite ainsi aucun chemin spécifique à ResNet : le modèle produit sa sortie via la même interface de propagation avant, puis transmet sa liste de paramètres à l'optimiseur. Cette conception permet d'intégrer la structure résiduelle dans le système actuel sans réécrire le framework d'entraînement, tout en vérifiant directement sa compatibilité avec le chemin principal d'autodiff.

5.8.1 Structure et stratégie de sous-échantillonnage

La configuration par défaut de `ResNetConfig` vise des entrées MNIST, avec une forme d'entrée $1 \times 28 \times 28$ et 10 classes. Le réseau commence par une couche de convolution de type stem, puis enchaîne trois stages ; leurs nombres de canaux sont respectivement 16, 32 et 64, et chaque stage contient deux blocs résiduels. Les dimensions spatiales évoluent selon $28 \times 28 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$, avant une couche entièrement connectée qui produit des logits de dimension 10.

5.8.2 BasicBlock

L'unité de base de l'implémentation ResNet actuelle est `BasicBlock`. Chaque bloc contient deux convolutions 3×3 :

$$x \rightarrow \text{Conv}_{3 \times 3} \rightarrow \sigma \rightarrow \text{Conv}_{3 \times 3} \rightarrow + \rightarrow \sigma,$$

où σ désigne la fonction d'activation configurée. Si les nombres de canaux d'entrée et de sortie sont identiques et que le stride vaut 1, le shortcut réutilise directement l'entrée ; sinon, le bloc ajoute une convolution de projection 1×1 afin d'aligner le nombre de canaux et les dimensions spatiales du shortcut sur la sortie de la branche principale.

Le calcul d'un bloc résiduel peut alors s'écrire :

$$F(x) = W_2 \sigma(W_1 x),$$

$$y = \sigma(F(x) + P(x)),$$

où $P(x)$ représente le chemin shortcut. Lorsque la forme ne change pas, $P(x) = x$; en cas de sous-échantillonnage ou de changement du nombre de canaux, $P(x)$ est assuré par une convolution 1×1 . Le passage d'un stage au suivant utilise une convolution de stride 2 pour le sous-échantillonnage, accompagnée d'une projection 1×1 sur le chemin shortcut afin de conserver la cohérence des dimensions spatiales et des canaux entre les deux branches.

5.8.3 Résumé

L'implémentation ResNet actuelle vise surtout à vérifier que la structure résiduelle s'intègre de manière stable au framework de différentiation automatique existant, tout en fournissant au projet une base CNN plus profonde que LeNet. Pour viser de meilleures performances sur des jeux de données plus complexes, il faudra encore ajouter BatchNorm, un global average pooling et une configuration d'entraînement plus systématique.

6 Recherche automatique d'hyperparamètres avec Optuna

En plus du réglage manuel, le projet a introduit **Optuna** pour organiser une recherche automatique d'hyperparamètres. Le point de départ est simple : les premières expériences sur MNIST ont déjà montré que le taux d'apprentissage, la taille de batch, la fonction d'activation et l'optimiseur influencent fortement la convergence. Avec l'ajout du chemin CNN, une exploration purement manuelle devient rapidement coûteuse. Le HPO est donc traité ici comme une extension directe du flux d'entraînement, et non comme un script auxiliaire ajouté après coup.

En pratique, chaque trial exécute un entraînement indépendant et prend `test_acc` comme objectif. La métrique d'évaluation peut être soit la meilleure précision de test observée pendant l'entraînement, soit la précision de test du dernier epoch :

$$\text{score} = \begin{cases} \max_e \text{test_acc}_e, & \text{metric=best,} \\ \text{test_acc}_E, & \text{metric=last.} \end{cases}$$

Le **study** associé est donc formulé en maximisation.

L'espace de recherche couvre principalement les paramètres d'entraînement les plus sensibles : le taux d'apprentissage est échantillonné sur une échelle logarithmique dans $[10^{-4}, 2 \times 10^{-2}]$, la taille de batch est choisie dans $\{32, 64, 128\}$, la fonction d'activation est comparée entre `relu`, `leaky_relu`, `gelu` et `tanh`, et l'initialisation alterne entre `he` et `xavier`. La partie optimiseur suit une recherche conditionnelle : `sgd` ne balaie que `weight_decay`, `momentum_sgd` ajoute `momentum` et `nesterov`, et `adamw` explore en plus `beta1`, `beta2` et `eps`. Cette organisation évite des combinaisons peu utiles et concentre le budget de trials sur des régions de paramètres plus pertinentes.

Les résultats expérimentaux sont sauvegardés avec la configuration des paramètres et le contexte d'exécution afin de faciliter la relecture et la reproduction. L'exécution prend aussi en charge le parallélisme intra-processus ainsi qu'un mode MPI worker pour augmenter le débit de recherche. Parmi les résultats CNN HPO conservés, une campagne de 40 trials avec 3 epochs par trial atteint best `test_acc = 0.9891`. Ce résultat indique que la recherche automatique peut identifier des configurations légèrement différentes des réglages manuels, mais plus efficaces sous un budget d'entraînement court.

7 Portage vers Tiny-ImageNet et pipeline d'entrée

Avec l'intégration du chemin CNN et de l'entraînement distribué, le champ expérimental s'étend de MNIST à **Tiny-ImageNet**. Ce portage augmente non seulement l'échelle des entrées, mais introduit aussi une charge système plus complète, avec décodage JPEG, entraînement convolutif et synchronisation distribuée ; il expose donc plus directement le domaine de validité et les goulots d'étranglement de l'implémentation actuelle.

7.1 Motivation du portage et caractéristiques des données

MNIST suffit pour valider les chemins de base liés au MLP, à la fonction de perte et à la différentiation automatique, mais ne couvre pas des scénarios plus lourds comme le chargement d'images JPEG, le calcul CNN ou la synchronisation MPI. À l'inverse, **Tiny-ImageNet** fournit environ 100000 images d'entraînement et environ 10000 images de validation, ce qui convient mieux à l'évaluation d'un pipeline d'entrée plus complexe et d'un flux d'entraînement plus chargé. Les expériences actuelles utilisent une séparation entre jeu d'entraînement et jeu de validation, sans jeu de test indépendant intégré au chemin principal.

7.2 Organisation du pipeline d'entrée

Pour s'adapter aux entrées image de **Tiny-ImageNet**, le système abandonne le schéma unique utilisé sur MNIST, fondé sur un chargement intégral suivi d'un découpage par index. Une abstraction **BatchSource** est introduite entre le loader et la source de données bas niveau afin de décrire, via une interface unifiée, la taille de l'échantillon, les dimensions d'entrée et le chargement d'un batch par index. Dans le cas de MNIST, cette abstraction peut se réduire à la copie de quelques lignes depuis une matrice déjà chargée; dans le cas de **Tiny-ImageNet**, elle devient un chargement et un décodage JPEG par batch pendant l'entraînement. Le loader réutilise les tampons de batch et conserve une logique unifiée pour les index et le mélange aléatoire.

7.3 État actuel de l'entraînement et principales limites

Le chemin **Tiny-ImageNet** est maintenant raccordé à la logique de chargement et à la boucle d'entraînement. Les évaluations mentionnées dans cette section correspondent à une configuration d'entraînement/évaluation avec séparation entre jeu d'entraînement et jeu de validation.

La figure 7 montre que la précision d'entraînement continue de monter jusqu'à 47%, alors que la précision de test commence à décroître dès le 5^e epoch et ne vaut plus que 7.24% au 40^e. Dans la configuration actuelle, avec une structure CNN de type LeNet et sans augmentation de données, le modèle présente donc un surapprentissage marqué sur **Tiny-ImageNet**.

7.4 Périmètre actuel et suite du travail

À ce stade, ce chemin permet déjà de charger **Tiny-ImageNet** et d'exécuter l'entraînement de manière stable, mais des limites nettes subsistent à la fois du côté des entrées et du côté du modèle. Le pipeline d'entrée n'intègre pas encore de prélecture asynchrone, de chevauchement avec le décodage, de mécanisme de cache ni d'augmentation de données plus complète; du côté du modèle et des opérateurs, le coût de réagencement entre `im2col/col2im`, le backward de la convolution ainsi que les transferts entre convolution et GEMM doivent encore être examinés. La suite du travail doit donc progresser en parallèle sur le système d'entrée et sur l'entraînement du modèle. Des entraînements de ResNet et d'IST sur cluster sont prévus afin d'améliorer la précision de test sur **Tiny-ImageNet**.

8 Entraînement distribué, synchronisation des gradients et réorganisation du runtime

8.1 Parallélisme de données synchrone et agrégation des gradients

L'entraînement distribué actuel repose sur un parallélisme de données synchrone : chaque `rank` conserve une copie complète du modèle et n'exécute les passes avant et arrière que sur sa

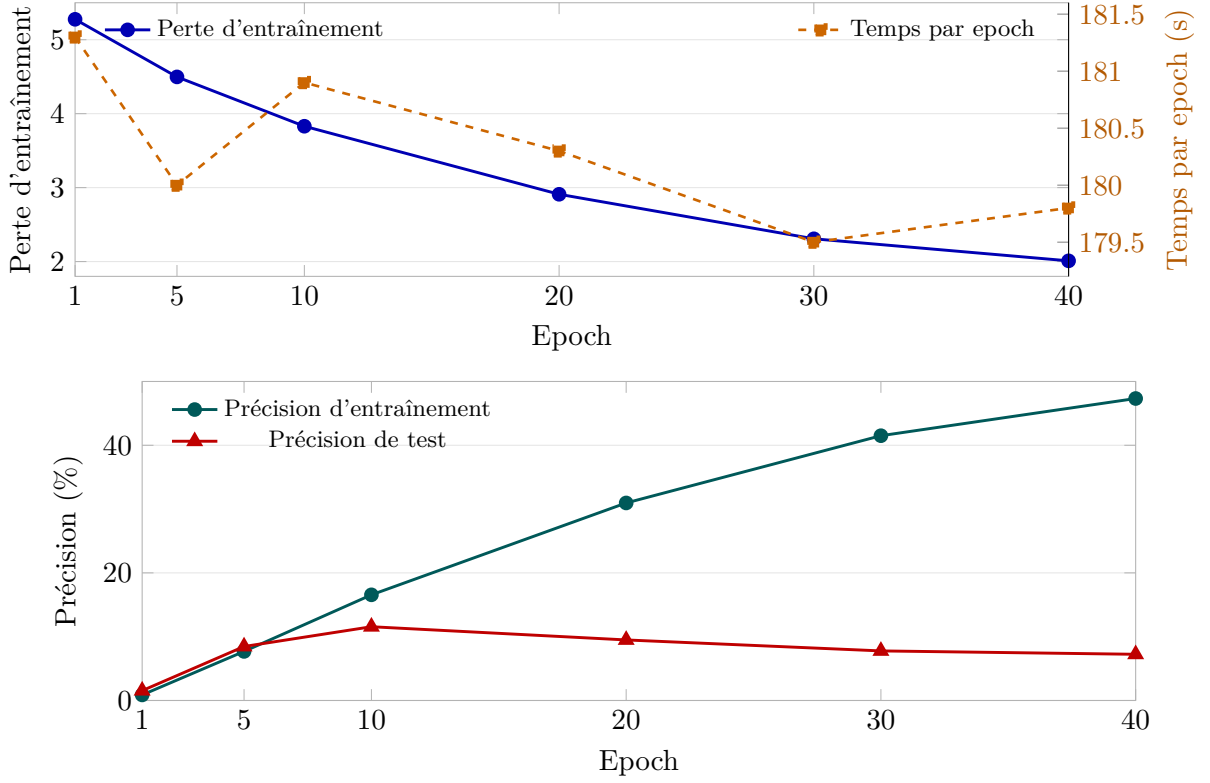


FIGURE 7 – Résumé des courbes d'entraînement sur 40 epochs pour Tiny-ImageNet (`world_size=4`, `batch=128`, synchronisation `per_param`). Le graphique du haut montre la perte d'entraînement et le temps par epoch ; celui du bas présente la précision d'entraînement et la précision de test.

propre partition locale de données. Si l'on note g_r la somme des gradients obtenue par le rank r à un step donné, le gradient global est d'abord obtenu par réduction :

$$g_{\text{sum}} = \sum_{r=0}^{\text{world_size}-1} g_r.$$

La taille globale du batch s'écrit alors

$$B_{\text{global}} = \sum_{r=0}^{\text{world_size}-1} B_r.$$

Le gradient moyen global utilisé pour la mise à jour des paramètres est donc

$$g_{\text{global}} = \frac{1}{B_{\text{global}}} g_{\text{sum}} = \frac{1}{B_{\text{global}}} \sum_{r=0}^{\text{world_size}-1} g_r.$$

Tous les ranks utilisent ensuite ce même gradient moyen pour mettre à jour les paramètres ; les copies du modèle restent donc cohérentes après chaque step. La difficulté principale ne tient pas à la définition mathématique, mais à l'organisation de la synchronisation des gradients. Une réduction bloquante paramètre par paramètre introduit de nombreux échanges de petite taille et ne constitue pas une solution durable.

8.2 Du paramètre par paramètre au gradient bucketing

Pour réduire les coûts fixes de communication, le système introduit ensuite le gradient bucketing : plusieurs gradients de paramètres sont d’abord empilés dans un tampon contigu, puis la réduction est effectuée à l’échelle du bucket. La sémantique de l’entraînement ne change pas ; seule la granularité de synchronisation passe du paramètre individuel au groupe de paramètres, ce qui diminue le nombre d’appels de communication collective.

8.3 Chevauchement entre communication et calcul

Une fois le bucketing stabilisé, le système ajoute un chemin de chevauchement entre communication et calcul : dès que tous les gradients d’un bucket sont prêts, une réduction asynchrone peut être lancée avant la fin complète du backward. Le moment de synchronisation est ainsi avancé, depuis « exécution groupée après le backward » vers « exécution dès disponibilité des gradients », ce qui ouvre la possibilité de masquer une partie de l’attente liée à la communication.

8.4 Compression des gradients : chemin d’implémentation et résultats intermédiaires

L’objectif de la compression des gradients est de réduire le coût de synchronisation tout en conservant autant que possible une convergence stable et une précision finale proche de la référence. Le premier chemin intégré réalisait une compression locale côté trainer, c’est-à-dire après la synchronisation des gradients et avant `optimizer.step`. Cette méthode modifie bien la représentation du gradient, mais ne réduit pas le volume de données échangé par `AllReduce` ; elle apporte donc peu de gain côté communication. Ce constat a conduit à déplacer la compression jusqu’à la frontière de communication.

Dans cette itération, la logique de compression a été déplacée dans le runtime de synchronisation distribuée, selon le schéma

`compress` → `AllReduce` → `decompress` → `step`

comme cadre d’exécution, intégré de manière uniforme aux chemins `bucketed` et `overlap_bucketed`. L’implémentation associée ajoute des hooks d’encodage/décodage et des entrées de profiling décomposées pour distinguer les coûts d’encodage, de communication et de décodage ; le premier noyau de quantification, trop coûteux, a ensuite été remplacé par une implémentation plus légère fondée sur des opérations bit à bit. En parallèle, la stratégie de déclenchement de la compression a été corrigée : elle ne suit plus un intervalle entre buckets, mais un intervalle entre steps. Sur la plateforme actuelle, `interval=8` fournit un compromis satisfaisant entre coût ajouté et bénéfice observé.

Sous `world_size=2`, sur MNIST, avec MLP(128), `batch=64` et `seed=42`, les médianes obtenues sur trois répétitions sont les suivantes :

- **bucketed** : pour `fp16+i8` par rapport à la baseline, `avg_step_time_ms` vaut +1.84%, `samples_per_s` vaut -1.80%, et `test_acc` vaut -0.02% ;
- **overlap_bucketed** : pour `fp16+i8` par rapport à la baseline, `avg_step_time_ms` vaut +1.58%, `samples_per_s` vaut -1.54%, et `test_acc` vaut -0.02%.

Au vu de ces résultats, la configuration retenue à ce stade est : `--grad_compress 1 --grad_compress_mode fp16 --grad_compress_interval 8`.

Même si le chemin de compression à la frontière de communication est désormais en place et que son surcoût est mieux contrôlé, le payload de communication utilise encore des tampons en `float`. Les résultats rapportés ici couvrent donc surtout la validité du cadre de compression et de la stratégie d’ordonnancement ; ils ne signifient pas encore que le volume d’octets communiqué a déjà fortement diminué. La suite du travail doit porter sur une communication réellement

compressée du payload, puis sur une nouvelle évaluation de son gain de bout en bout avec la même décomposition des mesures.

8.5 Expérience de montée en charge MPI

Le tableau 3 présente les résultats obtenus sur le CNN MNIST avec `local_batch=64` et `seed=42` lorsque le nombre de processus varie. Dans cette configuration, `local_batch` est fixé pour chaque rank ; `global_batch` augmente donc avec `np`.

Nb processus	local batch	global batch	Précision en test	Temps (s)	Speedup relatif
1	64	64	93.79%	31.66	1.00×
2	64	128	91.93%	19.49	1.62×
4	64	256	88.63%	11.98	2.64×

TABLE 3 – Résultats de montée en charge multi-processus pour le CNN MNIST (1 epoch, `local_batch=64`, `seed=42`)

Lorsque le nombre de processus passe de 1 à 4, le temps d'exécution diminue de 31.66s à 11.98s, soit un speedup de 2.64×. La variation de précision observée dans ce tableau dépend à la fois de l'augmentation de `global_batch` et de la réduction du nombre de mises à jour par epoch.

Pour mieux comprendre ce qui limite le gain de montée en charge, la suite examine les résultats de profiling décomposés.

8.6 Résultats de profiling : décomposition des temps de communication

Les résultats de profiling présentés ici servent principalement à montrer comment se décomposent les postes `fwd_bwd`, `sync_total` et `sync_wait`, et non à établir une hiérarchie strictement comparable entre les différents modes.

Mode de synchronisation	fwd_bwd (s)	sync_total (s)	sync_wait (s)
bucketed	7.57	1.60	1.60
overlap_bucketed	8.25	1.09	0.97

TABLE 4 – Extrait illustratif des postes de temps dans le profile de synchronisation

`overlap_bucketed` reste, à ce stade, dans un périmètre de validation de correction. Les résultats fournis ici servent donc surtout à localiser les coûts de communication et d'attente ; le gain réel lié au chevauchement devra être vérifié séparément dans une configuration strictement alignée.

8.7 Autograd : du backward implicite vers un moteur d'exécution explicite

Le chevauchement entre calcul et communication dépend d'un événement runtime plus fin : le moment exact où le gradient d'un paramètre devient disponible. L'ancienne version de l'autodiff savait accumuler les gradients, mais ne rendait pas cet instant explicite. Elle supportait donc mal des mécanismes comme les *ready hooks* au niveau des paramètres, la détection de disponibilité des buckets et la synchronisation chevauchée.

8.8 Refonte structurelle d'un Engine explicite

Pour cette raison, le projet a regroupé l'ordonnancement du backward dans un **AutogradEngine** explicite : le moteur maintient de façon centralisée le sous-graphe atteignable en sens inverse, le comptage des contributions de gradient et la *ready queue*, puis déclenche le calcul backward d'un noeud lorsque tous les gradients de sortie nécessaires sont disponibles. En parallèle, la logique backward des opérateurs a été réécrite pour suivre un schéma « sauvegarder le contexte et renvoyer les contributions de gradient », au lieu d'écrire directement les gradients dans les noeuds parents.

Le rôle central de cette refonte est de rendre explicite l'événement **parameter-ready**. Le runtime distribué peut alors y raccorder le packaging des gradients, les réductions asynchrones et le profiling associé, sans modifier les formules backward propres à chaque couche.

8.9 Réduction du coût sur le chemin chaud d'Autograd

Après cette refonte, l'optimisation de **AutogradEngine** reste limitée au coût de gestion du runtime. Trois points sont principalement visés : densifier localement l'indexation de l'état du sous-graphe backward atteignable afin de réduire les coûts de recherche ; limiter les allocations de petits objets pour diminuer le surcoût de gestion du framework ; et faire converger autant que possible la fusion des gradients vers des accumulations en place sur des tampons contigus.

Le profiling actuel indique que la part de runtime propre à **AutogradEngine** a déjà diminué. Les points chauds restants se concentrent davantage sur les opérateurs liés à la convolution, en particulier `im2col`, le GEMM de la passe avant, les GEMM backward `dX/dW` et `col2im`.

Références

- [1] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-based learning applied to document recognition.” *Proceedings of the IEEE*, 86(11) :2278–2324, 1998.
- [2] Jack J. Dongarra et al. “A set of level 3 basic linear algebra subprograms.” *ACM Transactions on Mathematical Software*, 16(1) :1–17, 1990.

Glossaire

A Environnement expérimental GEMM, workloads et protocole de recherche

A.1 Environnement expérimental

A.1.1 Plateforme matérielle

Composant	Spécification
Processeur	AMD Ryzen 9 8940HX, architecture x86_64
Cœurs physiques / threads logiques	16 cœurs physiques, 32 threads logiques (2 threads par cœur, SMT)
Restriction expérimentale	Limitation à 8 CPU via <code>taskset -c 0-7</code> , avec vérification du mapping sur les cœurs physiques par <code>lscpu -e=CPU,CORE,SOCKET,NODE</code>
Hierarchie de cache	L1d 32 KiB/cœur, L2 1 MiB/cœur, L3 64 MiB (partagé)
Fréquence d'exécution	Verrouillée à 4.0 GHz (Turbo Boost désactivé)
Mémoire	30 GiB

TABLE 5 – Configuration matérielle des expériences GEMM

A.1.2 Environnement logiciel

Composant	Version / détails
Système d'exploitation	Fedora Linux 42
Compilateur	GCC 15.2.1, options de compilation <code>-O3 -march=native</code>
Bibliothèque BLAS	OpenBLAS (version système)
Modèle parallèle	OpenMP 5.0
Jeu d'instructions vectorielles	AVX2 / AVX-512 (activé automatiquement via <code>-march=native</code>)

TABLE 6 – Configuration logicielle des expériences GEMM

A.1.3 Protocole de mesure

Le protocole de mesure est le suivant :

- **Préchauffage (warm-up)** : 5 itérations de préchauffage sont exécutées avant chaque série afin de stabiliser l'état des caches d'instructions et de données ;
- **Mesure de temps** : le programme exécute 2000 itérations en boucle interne, chronométrées avec `std::chrono::high_resolution_clock` ;
- **Compteurs matériels** : `perf stat -r 3` répète la mesure 3 fois au niveau externe et collecte le nombre d'instructions, de cycles, de commutations de contexte, de migrations CPU et de défauts de cache ;
- **Isolation de l'environnement** : les séries critiques sont séparées par un redémarrage du système, un vidage du page cache (`drop_caches`) et un verrouillage de la fréquence CPU via `cpupower`, afin de réduire l'hétérogénéité de l'état système. Le préchauffage reste le principal mécanisme de stabilisation du chemin d'exécution ; le redémarrage et le vidage du page cache servent uniquement à limiter les interférences entre séries.

Le modificateur `:u` (mode utilisateur) de `perf` exclut les commutations de contexte en mode noyau. Le protocole retient donc systématiquement `context-switches` sans modificateur afin de capturer l'ensemble des événements d'ordonnancement. Par ailleurs, les événements AMD uncore L3 sont collectés en mode system-wide et restent sensibles aux processus d'arrière-plan sur des fenêtres courtes ; ils ne sont donc pas retenus pour la comparaison quantitative. Le rapport utilise à la place `cache-misses` et `cache-references` en mode per-task comme indicateurs agrégés de pression mémoire. `cache-misses` sert uniquement à la comparaison relative entre configurations, sans attribution précise à L1, L2 ou L3.

Marge CPU et isolation expérimentale. Les expériences GEMM sont limitées à un budget contrôlé de cœurs CPU, sans occupation complète de tous les threads logiques, afin de laisser une marge au système d'exploitation, au runtime OpenMP, à la collecte `perf` et aux services d'arrière-plan, et de réduire les migrations de threads, les interruptions et la contention non contrôlée. Deux familles de configuration sont retenues : les configurations recommandées, limitées à un budget validé de cœurs physiques, et une configuration de stress à 16 threads avec SMT / hyper-threading.

A.2 Workloads utilisés pour le réglage mono-thread

Quatre familles de workloads représentatives sont définies à partir des formes de couches réellement utilisées dans le MLP et le CNN du projet :

Famille de workload	Couche source	M	K	N
fc_forward_mainstream	MLP FC1 (hidden=128)	32	784	128
fc_head_small_n	MLP FC2 (output)	32	128	10
conv_dx_skinny_k	CNN Conv2 backward	3200	16	150
fc_wide_output	MLP FC1 (hidden=256)	32	784	256

TABLE 7 – Workloads représentatifs utilisés pour le réglage mono-thread (batch=32)

A.3 Protocole de recherche en deux tours

Premier tour (filtrage global des candidats) : pour les 6 noyaux AVX2, plusieurs combinaisons (M_c, N_c, K_c) sont balayées sur chaque workload, soit 272 candidats au total. Chaque candidat est exécuté 1000 fois et évalué à partir du temps médian, avec en complément 5 échantillons `perf stat` pour les compteurs matériels. Le balayage suit une stratégie par étapes : K_c est d'abord exploré à M_c, N_c fixés, puis M_c avec le meilleur K_c , puis N_c dans une grille plus fine, de sorte que les variations de performance restent attribuables à un seul axe paramétrique.

Second tour (raffinement local) : les 5 combinaisons `KernelShape/Kc` les plus performantes sur les principales formes FC au premier tour sont conservées : `8x8/Kc=288`, `8x8/Kc=384`, `4x16/Kc=192`, `4x16/Kc=256`, `4x16/Kc=320`, et la grille de recherche en M_c, N_c est densifiée autour de leur meilleure région (environ 75 combinaisons au total) afin de localiser plus précisément les paramètres optimaux.

B Diagnostic des coûts d'ordonnancement et de synchronisation en configuration 16 threads

Ce diagnostic sert uniquement à fixer le protocole expérimental et ne fait pas partie des conclusions principales sur la performance.

L'élément central de ce contrôle provient d'une expérience d'affinité CPU (OpenBLAS, 16 threads, matrice 128×128 , 5000 itérations) : en ordonnancement libre, on mesure 2,013 commutations de contexte et 13.45 B instructions ; avec `OMP_PROC_BIND=true`, `OMP_PLACES=cores`, les commutations de contexte tombent à 14 (−99.3%) et le nombre total d'instructions à 4.05 B (−69.9%). Cette observation concerne les surcoûts de la configuration 16 threads, dominés ici par l'ordonnancement et l'attente de synchronisation plutôt que par du calcul supplémentaire.

Indicateur	Libre	Cœurs liés	Variation
context switch (CS)	2,013	14	−99.3%
Instructions totales	13.45 B	4.05 B	−69.9%
Écart-type cache misses	1.32%	0.06%	stabilité multipliée par ~ 22

TABLE 8 – Effet de l'affinité CPU sur la performance à 16 threads (matrice 128×128 , 5000 itérations)

Les expériences suivantes sont donc limitées à 1 à 8 threads physiques avec liaison explicite aux cœurs, afin d'éviter que la variabilité d'ordonnancement propre au SMT n'interfère avec la comparaison des backends.

La figure 8 regroupe la comparaison de passage à l'échelle après contrainte d'affinité.

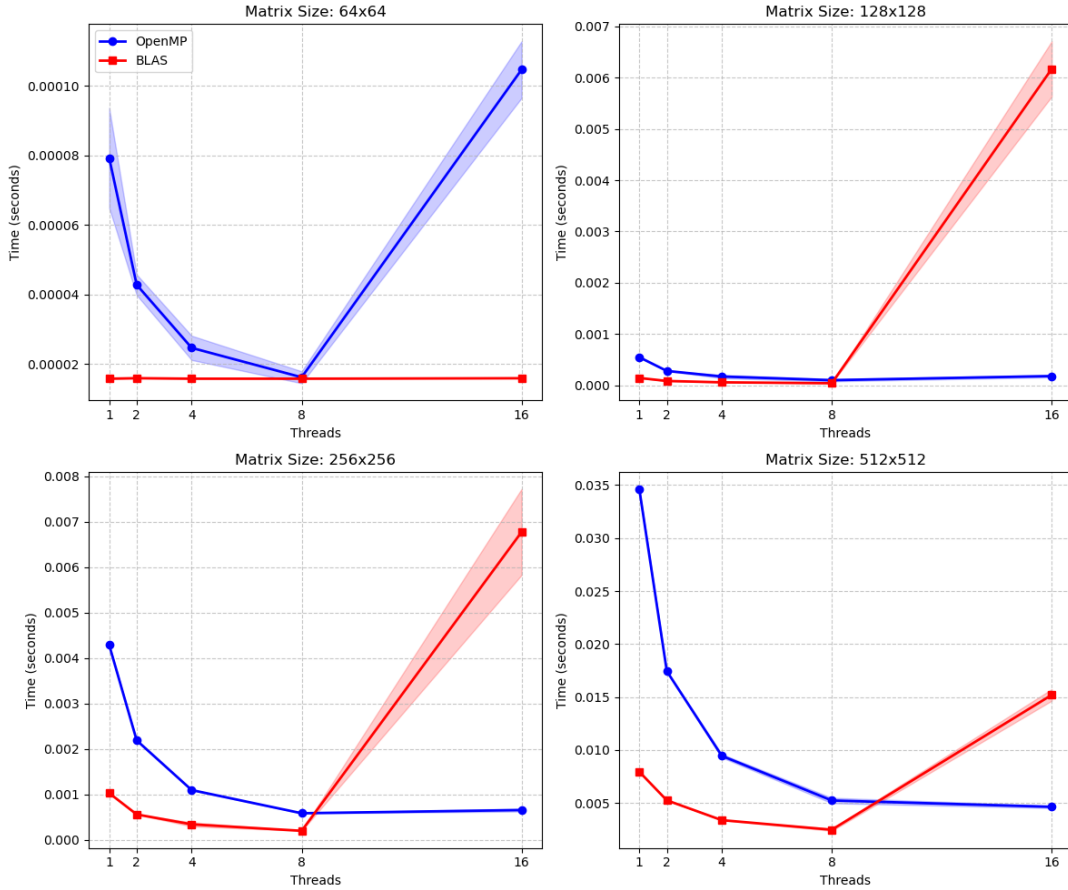


FIGURE 8 – Comparaison de la scalabilité en nombre de threads après contrainte d'affinité CPU (BLAS2). Sur les matrices carrées de 64×64 à 512×512 couvertes par la figure, et avec 8 threads physiques, `blas` fournit le temps mesuré le plus faible parmi les implémentations comparées.

C Le filtrage des paramètres OpenMP blocked / packing

Cette annexe complète le protocole intégral des expériences de la famille blocked présentées après la section 2.2. Les essais portent sur trois variantes : `omp_blocked`, `omp_blocked_packb` et `omp_blocked_packab`. Le filtrage se déroule en deux tours : le premier effectue une sélection de paramètres sur les petites matrices carrées 64, 128, 256 et 512, avec 1, 2, 4 et 8 threads physiques et 1000 répétitions par défaut ; le second réévalue sur 1024 et 2048 le petit nombre de candidats conservés afin de vérifier leur stabilité sur grandes matrices et d'en déduire les paramètres représentatifs.

L'espace de candidats du premier tour n'est pas issu d'une recherche libre complète ; il est construit à partir des contraintes théoriques de résidence en cache données dans le corps du texte. `omp_blocked` teste `BlockSize` $\in \{16, 32, 48, 64, 128, 256\}$, avec les 3 premières valeurs orientées L1 et les 3 suivantes orientées L2 ; `omp_blocked_packb` se limite à `BlockSize` $\in \{16, 32, 48\}$, car l'objectif est d'évaluer une stratégie packed- B orientée L1 ; `omp_blocked_packab` n'utilise plus une taille de bloc unique, mais retient parmi `PackM`, `PackN` $\in \{32, 48, 64, 96, 128\}$ et `PackK` $\in \{16, 32, 48, 64, 96\}$ les combinaisons compatibles avec la contrainte de footprint L1, puis applique le filtrage du premier tour sur cet ensemble.

À l'issue du premier tour, le programme agrège d'abord les résultats, puis sélectionne automatiquement les quelques candidats du second tour à partir du temps moyen inter-threads sur une taille de référence, par défaut la plus grande taille filtrée, soit 512. La figure 9 présente uniquement la synthèse moyenne par thread du premier tour pour `omp_blocked` sur les quatre tailles carrées ; elle sert à situer les block sizes les uns par rapport aux autres, sans désigner directement un optimum unique. Les synthèses de candidats et les mesures de revalidation du second tour sont conservées pour vérification ultérieure.

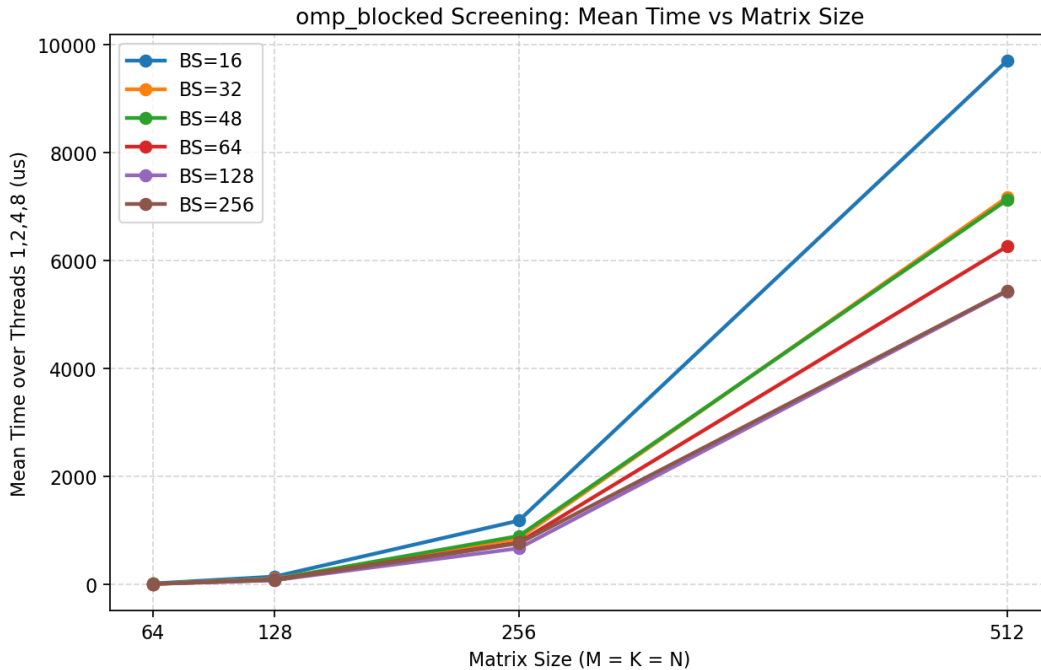


FIGURE 9 – Synthèse du filtrage sur petites matrices pour `omp_blocked` entre 64 et 512. Après moyenne des résultats à 1, 2, 4 et 8 threads physiques, `BlockSize=64` apparaît plus souvent dans la zone favorable sur les matrices les plus grandes, sans optimum unique commun à toutes les tailles de matrices et à tous les nombres de threads.

Le CSV de synthèse `omp_blocked_packb` associé à la figure 9 indique que, dans l'intervalle

conservé `BlockSize` $\in \{16, 32, 48\}$, les valeurs 32 et 48 dominent de façon assez stable 16 ; sur 64×64 , l'avantage revient le plus souvent à 32, sur 128×128 à 48, et sur 256×256 comme sur 512×512 , la tête alterne fréquemment entre 32 et 48. Pour `omp_blocked_packab`, les candidats du second tour se concentrent après filtrage sur un petit nombre de combinaisons orientées L1, et les paramètres représentatifs finalement retenus sont $(\text{PackM}, \text{PackN}, \text{PackK}) = (32, 128, 16)$.

D Dérivation de l'ensemble de candidats AVX2 pour les blocked sizes

Cette annexe complète la justification de la réduction de l'espace de recherche $(M_r, N_r, K_c, M_c, N_c)$ présentée dans le corps du rapport.

D.1 Dérivation de m_r et n_r liée à l'implémentation

Le micro-kernel courant exécute à chaque itération p $C_r(i, :) \leftarrow C_r(i, :) + A_r(i, p) B_r(p, :)$, où $0 \leq i < m_r$. Ici, $B_r(p, :)$ est chargé d'un seul coup dans un registre vectoriel AVX2, tandis que $A_r(i, p)$ participe au FMA par diffusion scalaire. Le bloc sémantique $m_r \times n_r$ reste inchangé ; les lanes SIMD correspondent à la dimension n_r , ce qui donne une orientation des registres différente de celle de certains exemples de la littérature.

On note N_{vec} le nombre de scalaires contenus dans un registre vectoriel. Pour AVX2 en `float32`, on a $N_{\text{vec}} = \frac{256}{32} = 8$. Dans le cadre de l'optimisation paramétrique, n_r doit au minimum respecter la granularité de vectorisation et se prend généralement comme multiple entier de N_{vec} , soit $n_r = t N_{\text{vec}}$ avec $t \in \mathbb{Z}_{\geq 1}$. Si un même $B_r(p, :)$ est porté par deux registres YMM au cours de la même itération, n_r peut aussi valoir 16. La suite ne considère que $t \in \{1, 2\}$, donc $n_r \in \{8, 16\}$, sans traiter pour l'instant des micro-kernels plus larges en direction n , tels que $n_r = 24, 32$.

Les contraintes sur m_r proviennent du masquage de latence et du budget de registres. La condition de base issue des travaux BLIS sur les micro-kernels peut s'écrire $m_r n_r \geq N_{\text{vec}} L_{\text{vfma}} N_{\text{vfma}} = 8 \times 4 \times 2 = 64$, où L_{vfma} est la latence dépendante du FMA vectoriel, ici fixée à 4 pour un modèle de premier ordre, et N_{vfma} le nombre de FMA vectoriels émis par cycle, ici fixé à 2. Cette relation fournit la borne inférieure côté débit.

Le budget de registres ne peut plus s'écrire sous une forme unique indépendante de n_r , car lorsque $n_r = 8$, chaque ligne d'accumulateurs n'occupe qu'un YMM, alors que pour $n_r = 16$, chaque ligne en occupe 2. En posant $t = n_r / N_{\text{vec}} = n_r / 8$, le nombre de registres accumulateurs vaut $m_r t$. Le budget de registres doit donc au minimum dépendre de t : si une itération conserve seulement 1 registre temporaire vectoriel pour B , on obtient $m_r t + 1 + \delta \leq N_{\text{reg}} = 16$; et, avec un modèle plus conservateur pour $n_r = 16$, en considérant que les deux sous-vecteurs de B doivent être conservés simultanément, on obtient $m_r t + t + \delta \leq N_{\text{reg}} = 16$. où N_{reg} est le nombre de registres vectoriels disponibles, et δ la marge réservée au broadcast, à la génération d'adresses et à l'ordonnancement, avec $\delta \in [1, 3]$.

Il faut donc distinguer les cas. Quand $n_r = 8$, soit $t = 1$, la borne de débit donne $m_r \cdot 8 \geq 64 \Rightarrow m_r \geq 8$, tandis que le budget de registres optimiste donne $m_r + 1 + \delta \leq 16$. Pour $\delta \in [1, 3]$, on obtient $8 \leq m_r \leq 13$, d'où l'ensemble conservé dans ce modèle : $m_r \in \{8, 9, 10, 11, 12, 13\}$. Quand $n_r = 16$, soit $t = 2$, la borne de débit donne $m_r \cdot 16 \geq 64 \Rightarrow m_r \geq 4$. Le budget de registres optimiste devient $2m_r + 1 + \delta \leq 16$, et sa version plus conservatrice s'écrit $2m_r + 2 + \delta \leq 16$. Sous cet ensemble de contraintes, l'intervalle faisable de m_r est plus étroit pour $n_r = 16$; en retenant les candidats les plus sûrs, on obtient $m_r \in \{4, 5, 6\}$, $m_r = 7$ n'est conservable que dans l'hypothèse plus optimiste où $n_r = 16$ et un seul registre temporaire vectoriel de B est gardé.

Le domaine faisable de cette section se résume alors, selon les cas en $n_r/8$, à :

$$\begin{cases} n_r \in \{8, 16\}, \\ m_r n_r \geq 64, \\ m_r \frac{n_r}{8} + 1 + \delta \leq 16 & \text{(version optimiste ; pour } n_r = 16, \text{ un seul registre temporaire pour } B), \\ m_r \frac{n_r}{8} + \frac{n_r}{8} + \delta \leq 16 & \text{(version conservatrice ; pour } n_r = 16, \text{ tous les sous-vecteurs de } B). \end{cases}$$

On en déduit que les couples candidats (m_r, n_r) du domaine faisable initial s'écrivent

$$\{(8, 8), (9, 8), (10, 8), (11, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\},$$

où $(7, 16)$ ne reste possible que sous un budget de registres plus optimiste ; il n'est donc pas inclus dans l'ensemble de candidats par défaut. Pour contenir la taille de la recherche ultérieure, l'ensemble finalement retenu pour (m_r, n_r) est $\{(8, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\}$.

D.2 Modèle de cache et génération des candidats pour k_c

B.2.1 Centres candidats de k_c fondés sur L1

Pour un couple (m_r, n_r) donné, k_c fixe la profondeur de packed $A_r \in \mathbb{R}^{m_r \times k_c}$ et de $B_r \in \mathbb{R}^{k_c \times n_r}$. Le rapport construit d'abord des centres candidats à partir de l'occupation des panneaux côté L1, puis en déduit un ensemble discret à l'aide des contraintes d'alignement et de page footprint.

Pour AVX2 en `float32`, en prenant la cache line comme unité et 1024B par voie de L1, la pression approximative sur les sets des panneaux A_r et B_r s'écrit proportionnellement à $m_r k_c$ et $n_r k_c$. Après simplification, la borne supérieure du budget de sets disponible pour A_r est $C_{A_r}^{\max} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$, ce qui donne le centre candidat $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\max}}{m_r}$.

k_c^{center} n'est utilisé ici que comme centre candidat sur le matériel courant. Le micro-kernel lit d'abord $B_r(p, :)$ puis diffuse les scalaires de $A_r(:, p)$ à chaque itération p ; le modèle L1 sert donc, dans ce rapport, à fournir une valeur centrale et le point de départ de la discrétisation.

B.2.2 Contraintes sur k_c fondées sur le TLB / page footprint

La relation L1 / set-mapping ne suffit pas à elle seule pour fixer k_c , car l'augmentation de k_c accroît simultanément le page footprint des micro-panneaux et des macro-panneaux. Si la taille de page vaut P , le footprint au niveau micro-panneau vérifie approximativement

$$\frac{m_r k_c S_{\text{data}}}{P} \lesssim E_{A_r, \text{TLB}}, \quad \frac{k_c n_r S_{\text{data}}}{P} \lesssim E_{B_r, \text{TLB}},$$

Cette forme de filtrage reste indicative dans le rapport, qui passe ensuite à un filtrage au niveau macro-panneau. Pour rendre la couche TLB exploitable sous forme de règle, on utilise ici $m_c = 128$ et $n_c = 512$ comme volumes de panneaux provisoires ; ils servent uniquement au TLB-aware pruning de ce tour. Pour $P = 4096$, on obtient

$$\begin{aligned} \text{pages}(A_c) &\approx \left\lceil \frac{m_c k_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{128 \cdot k_c \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{8} \right\rceil, \\ \text{pages}(B_c) &\approx \left\lceil \frac{k_c n_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{k_c \cdot 512 \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{2} \right\rceil. \end{aligned}$$

Sur la machine étudiée, les L1 DTLB et L2 DTLB disposent respectivement de 72 et 3072 entrées. Comme le nombre de pages de B_c reste très inférieur à la capacité du L2 DTLB pour

les candidats de cette section, le filtrage réellement discriminant provient de A_c , privé à chaque thread. Afin de laisser une marge pour la pile, le bloc C , le code et les autres données, on retient le budget effectif conservateur $E_{A,\text{eff}} = 0.75 \times 72 = 54$, avec la règle de pruning suivante : si un candidat vérifie $\text{pages}(A_c) > 54$, il est retiré de l'ensemble expérimental du premier tour ; si $\text{pages}(A_c) \leq 54$ et $\text{pages}(B_c) \ll 3072$, il passe le contrôle de cette couche TLB. Cette couche n'assure qu'un filtrage approximatif et ne génère pas de nouveau centre.

B.2.3 Génération des candidats k_c sous contraintes L1 et TLB

La génération des candidats suit deux étapes. La première produit, à partir du modèle L1, un ensemble discret initial autour du centre candidat. En notant g_k la granularité d'alignement de l'implémentation pour k_c , on obtient

$$\mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}),$$

où g_k est déterminé par le pas du packed panel et la granularité des boucles externes en blocs, tandis que $\mathcal{A}_k$ désigne un voisinage multiplicatif fini autour de 1. Pour contenir la taille du premier tour, on prend ici l'instanciation conservatrice $g_k = 32$ et $\mathcal{A}_k = \{0.75, 1.00, 1.25\}$.

La seconde étape applique un filtrage approximatif à $\mathcal{K}_{L1}(m_r, n_r)$ au moyen des contraintes TLB / page footprint, ce qui donne l'ensemble expérimental du premier tour

$$\mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r).$$

Ici, k_c^{center} , $\mathcal{K}_{L1}$ et $\mathcal{K}_{\text{pruned}}$ dépendent tous du couple (m_r, n_r) considéré. Les expériences suivantes peuvent soit explorer séparément chaque (m_r, n_r) , soit fusionner les ensembles dans un pool commun, sans changer la hiérarchie : génération par L1, puis pruning par TLB.

Pour les 6 couples (m_r, n_r) finalement retenus à la section B.1, l'application de $C_{A_r}^{\text{max}} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$ et de $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\text{max}}}{m_r}$ conduit au tableau 9. Le tableau présente d'abord $\mathcal{K}_{L1}(m_r, n_r)$, puis $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ après filtrage par la couche TLB ; les candidats trop grands, pour lesquels le nombre de pages du A_c provisoire dépasse 54, sont effectivement supprimés.

(m_r, n_r)	$C_{A_r}^{\text{max}}$	k_c^{center}	$\mathcal{K}_{L1}$	$\mathcal{K}_{\text{pruned}}$
(8, 8)	3	384.0	{288, 384, 480}	{288, 384}
(12, 8)	4	341.3	{256, 352, 416}	{256, 352, 416}
(13, 8)	4	315.1	{224, 320, 384}	{224, 320, 384}
(4, 16)	1	256.0	{192, 256, 320}	{192, 256, 320}
(5, 16)	1	204.8	{160, 192, 256}	{160, 192, 256}
(6, 16)	1	170.7	{128, 160, 224}	{128, 160, 224}

TABLE 9 – Génération des candidats k_c et résultat du pruning TLB

Ces $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ constituent les candidats du premier tour pour la recherche sur k_c . Ils proviennent d'un centre calculé sur le matériel courant, d'un alignement à 32 et d'un voisinage multiplicatif fini, puis d'un filtrage page footprint / TLB ; par exemple, le $k_c = 480$ issu du facteur 1.25 est retiré sur la ligne (8, 8), car le nombre de pages du A_c provisoire atteint 60, au-dessus du budget effectif de 54 entrées L1 DTLB.

Les contraintes de cette section se résument à

$$\begin{cases} C_{A_r} + C_{B_r} \leq W_{L1} - 1, \\ C_{A_r} \leq \left\lfloor \frac{W_{L1} - 1}{1 + n_r/m_r} \right\rfloor, \\ k_c^{\text{center}} \approx \frac{1024 C_{A_r}}{m_r}, \\ \mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}), \\ \mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r). \end{cases}$$

D.3 Génération des candidats de base mono-thread pour m_c/n_c

B.3.1 Contraintes sur m_c/n_c fondées sur le cache

m_c et n_c sont construits à partir des candidats (m_r, n_r, k_c) déjà filtrés : une fois la forme du micro-kernel et la profondeur de panneau le long de k fixées, m_c détermine la hauteur du jeu de travail privé $A_c \in \mathbb{R}^{m_c \times k_c}$ et n_c la largeur du jeu de travail partagé $B_c \in \mathbb{R}^{k_c \times n_c}$. La présente section transforme ces contraintes de cache / TLB en ensembles de candidats de base pour le mono-thread ou le parallélisme faible.

Dans l'implémentation courante, A_c est traité comme jeu de travail privé à chaque thread, tandis que B_c correspond à un panneau partagé en lecture sur le chemin multithread ; la section raisonne donc d'abord sur leurs volumes au titre des candidats de base mono-thread. Leurs tailles en octets sont

$$|A_c| = m_c k_c S_{\text{data}}, \quad |B_c| = k_c n_c S_{\text{data}}.$$

La hiérarchie de cache conduit alors aux contraintes d'échelle les plus directes

$$m_c k_c S_{\text{data}} \lesssim \rho_A S_{\text{private}}, \quad k_c n_c S_{\text{data}} \lesssim \rho_B S_{\text{shared}}.$$

où $\rho_A, \rho_B \in (0, 1)$ sont des coefficients de modélisation destinés à conserver une marge pour les conflits, le code et les autres jeux de travail ; ces relations donnent les bornes d'échelle de base m_c^{max} et n_c^{max} . Sur la machine courante, le modèle mono-thread de base prend une capacité L2 privée par cœur $S_{\text{private}} = 1$ MiB ; pour borner le budget d'un panneau B_c , il retient aussi un budget effectif conservateur de cache partagé $\rho_B S_{\text{shared}} = 1$ MiB. Avec $\rho_A = 1/4$, on obtient

$$\begin{aligned} m_c^{\text{max}}(k_c) &\approx \frac{\rho_A S_{\text{private}}}{k_c S_{\text{data}}} = \frac{256 \text{ KiB}}{4k_c} = \frac{65536}{k_c}, \\ n_c^{\text{max}}(k_c) &\approx \frac{\rho_B S_{\text{shared}}}{k_c S_{\text{data}}} = \frac{1 \text{ MiB}}{4k_c} = \frac{262144}{k_c}. \end{aligned}$$

La couche cache sert donc ici à fournir les bornes d'échelle de base $m_c^{\text{max}}(k_c)$ et $n_c^{\text{max}}(k_c)$ en fonction de k_c , autour desquelles seront générés les candidats discrets.

B.3.2 Contraintes sur m_c/n_c fondées sur le TLB / page footprint

Les bornes de cache ne suffisent pas à elles seules, car la couverture en pages de A_c et B_c augmente avec m_c et n_c . Si la taille de page vaut P , leur couverture approchée est

$$\frac{m_c k_c S_{\text{data}}}{P}, \quad \frac{k_c n_c S_{\text{data}}}{P},$$

Les conditions approximatives de pruning côté TLB deviennent alors

$$\frac{m_c k_c S_{\text{data}}}{P} \lesssim E_{A, \text{TLB}}, \quad \frac{k_c n_c S_{\text{data}}}{P} \lesssim E_{B, \text{TLB}}.$$

Dans cette section, ces relations jouent le rôle d'une seconde couche de pruning. Le code courant ne contrôle ni huge pages, ni STLB, ni appartenance NUMA, ni affinité de thread ; elles restent donc utilisées comme contraintes approchées. Pour la machine étudiée, on prend $P = 4096$, 72 entrées en L1 DTLB, 3072 en L2 DTLB, et les budgets effectifs conservateurs $E_{A,\text{eff}} = 54$ et $E_{B,\text{eff}} = 192$; la valeur 192 sert à éliminer dès le premier tour les panneaux B_c trop larges. On obtient alors

$$\text{pages}(A_c) \approx \frac{m_c k_c}{1024}, \quad \text{pages}(B_c) \approx \frac{k_c n_c}{1024}.$$

La règle de TLB-aware pruning employée ici est donc la suivante : si un candidat m_c vérifie $\text{pages}(A_c) > 54$, il est retiré de $\mathcal{M}_{\text{cache}}$; si un candidat n_c vérifie $\text{pages}(B_c) > 192$, il est retiré de $\mathcal{N}_{\text{cache}}$. Le choix $E_{B,\text{eff}}$ reste inférieur à la capacité matérielle du L2 DTLB afin d'éliminer dès la recherche de base du premier tour les panneaux B_c trop larges.

B.3.3 Génération des candidats de base mono-thread sous contraintes cache et TLB

Pour un candidat (m_r, n_r, k_c) donné, les bornes de cache génèrent d'abord les ensembles discrets initiaux

$$\mathcal{M}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_m}^\downarrow(\{\beta m_c^{\max}(k_c) : \beta \in \mathcal{A}_m\}),$$

$$\mathcal{N}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_n}^\downarrow(\{\gamma n_c^{\max}(k_c) : \gamma \in \mathcal{A}_n\}),$$

puis un pruning TLB / page footprint produit

$$\mathcal{M}_{\text{pruned}} \subseteq \mathcal{M}_{\text{cache}}, \quad \mathcal{N}_{\text{pruned}} \subseteq \mathcal{N}_{\text{cache}}.$$

Pour contenir la taille de la recherche de base du premier tour en mono-thread, on prend $g_m = 32$, $g_n = 64$ et $\mathcal{A}_m = \mathcal{A}_n = \{0.5, 0.75, 1.0\}$. Ici, $\text{Align}^\downarrow$ désigne l'alignement vers le bas sur le multiple de granularité le plus proche qui ne dépasse pas la valeur cible. En appliquant cette même règle à toutes les combinaisons (m_r, n_r, k_c) conservées à la section B.2, on obtient les candidats de base mono-thread du tableau 10.

(m_r, n_r, k_c)	$m_c^{\max}$	$\mathcal{M}_{\text{cache}}$	$\mathcal{M}_{\text{pruned}}$	$n_c^{\max}$	$\mathcal{N}_{\text{cache}}$	$\mathcal{N}_{\text{pruned}}$
(8, 8, 288)	227.6	{96, 160, 224}	{96, 160}	910.2	{448, 640, 896}	{448, 640}
(8, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(12, 8, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(12, 8, 352)	186.2	{64, 128, 160}	{64, 128}	744.7	{320, 512, 704}	{320, 512}
(12, 8, 416)	157.5	{64, 96, 128}	{64, 96, 128}	630.2	{256, 448, 576}	{256, 448}
(13, 8, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}
(13, 8, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(13, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(4, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(4, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(4, 16, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(5, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(5, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(5, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(6, 16, 128)	512.0	{256, 384, 512}	{256, 384}	2048.0	{1024, 1536, 2048}	{1024, 1536}
(6, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(6, 16, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}

TABLE 10 – Résultat du pruning sur M_c/N_c pour les candidats de base mono-thread du premier tour

Les ensembles $\mathcal{M}_{\text{pruned}}$ et $\mathcal{N}_{\text{pruned}}$ du tableau 10 constituent la base de candidats pour le mono-thread ou le parallélisme faible. Ils suivent une structure à deux niveaux : le cache fournit l'échelle initiale, puis le TLB retire les points trop larges. Les expériences suivantes examinent ensuite comment resserrer encore ces candidats.

Le coarse screening mono-thread du premier tour couvre l'ensemble des combinaisons (m_r, n_r, k_c) conservées ; sur cette base, seules les 5 combinaisons **KernelShape/Kc** localement gagnantes sont prolongées au second tour. La grille plus dense utilisée pour M_c/N_c au second tour est donnée au tableau 11.

Noyau	m_r	n_r	k_c	Candidats M_c	Candidats N_c	Nb. de combinaisons
8x8	8	8	288	{96, 128, 160, 192}	{448, 512, 576, 640}	16
8x8	8	8	384	{64, 96, 128}	{320, 384, 448, 512}	12
4x16	4	16	192	{160, 192, 224, 256, 288}	{640, 768, 896, 1024}	20
4x16	4	16	256	{128, 160, 192}	{512, 576, 640, 704, 768}	15
4x16	4	16	320	{96, 128, 160}	{384, 448, 512, 576}	12

TABLE 11 – Grille de candidats M_c/N_c utilisée pour le filtrage fin du second tour

Le nombre total de candidats au second tour est donc $16 + 12 + 20 + 15 + 12 = 75$. Ces combinaisons ajoutent les points intermédiaires autour des régions déjà identifiées comme favorables au premier tour, ce qui renforce la stabilité des conclusions sur les blocked sizes en mono-thread.

E Synthèse des résultats de réglage AVX2 en mono-thread

Famille de workload	Meilleur noyau	M_c	N_c	K_c	Personnalisé (μs)	OpenBLAS (μs)	Écart
fc_forward (b32)	4x16	128	576	320	72.5	60.2	-17%
fc_forward (b64)	4x16	192	512	256	132.6	110.1	-17%
fc_head (b32)	4x16	—	—	—	4.76	1.14	-76%
conv_dx (b32)	8x8	160	640	288	744.3	647.3	-13%
fc_wide (b32)	4x16	128	768	256	136.5	122.2	-10%

TABLE 12 – Réglage AVX2 en mono-thread : meilleure configuration par famille de workloads et comparaison à OpenBLAS en mono-thread

F Complément sur la heatmap de réglage thread-aware $kCacheBlockM/N_c$

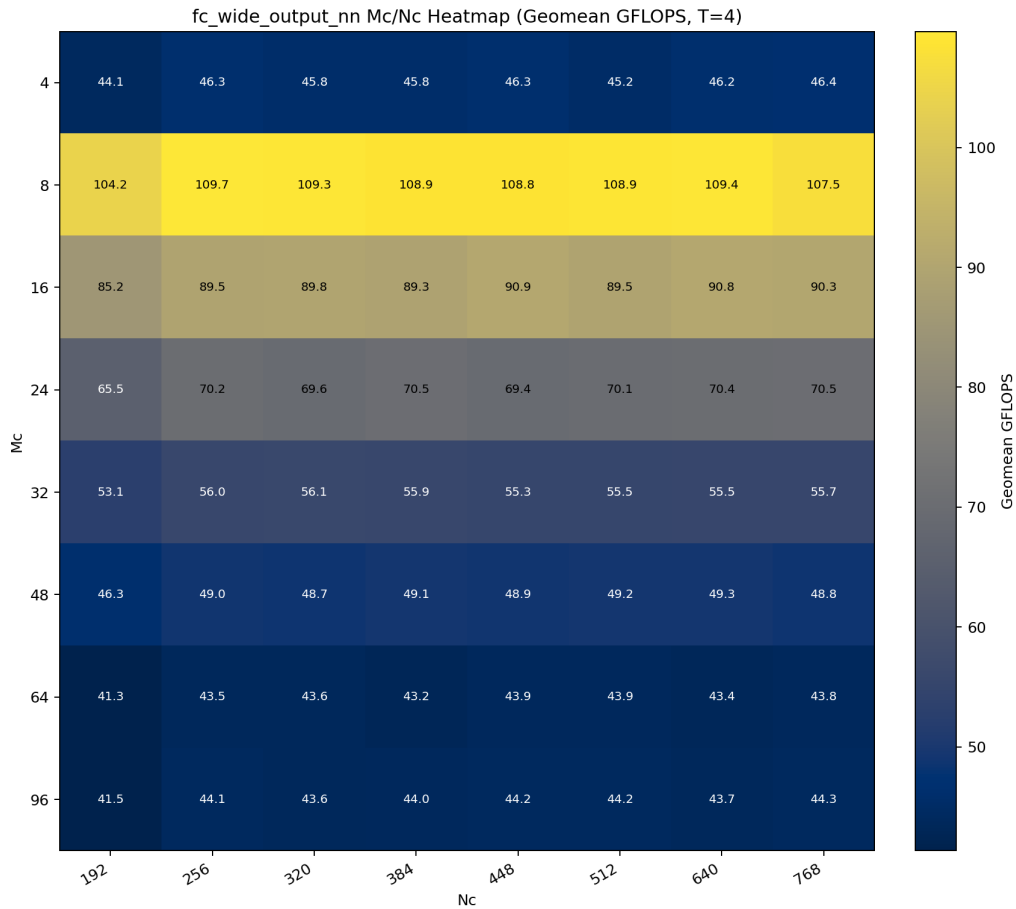


FIGURE 10 – Heatmap représentative du réglage thread-aware de $kCacheBlockM/N_c$ ($T = 4$, `fc_wide_output_nn`). La zone de haute performance se concentre surtout sur la ligne $kCacheBlockM = 8$ (soit $M_c = 64$ effectif), tandis que la direction N_c forme un plateau relativement plat.

G Tableau récapitulatif de strong scaling

Implémentation	Moyenne géométrique GFLOPS				Accélération à 8T	Efficacité parallèle à 8T
	1T	2T	4T	8T		
AVX2 GotoBLAS	26.1	41.7	64.4	84.4	3.23×	40.0%
AVX512 GotoBLAS	27.6	45.0	67.4	89.4	3.23×	40.0%
OpenBLAS	43.4	67.5	101.1	118.2	2.72×	34.0%

TABLE 13 – Expérience de strong scaling : moyenne géométrique des GFLOPS à chaque nombre de threads (sur 9 workloads)